



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Analizador de Mieles por
Espectrometría (HIVES)
Documentación Técnica**



Presentado por Diego Alonso Soria
en Universidad de Burgos — 9 de junio
de 2026

Tutores: Carlos Cambra Baseca y Rubén Ruiz
González

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	27
Apéndice B Especificación de Requisitos	33
B.1. Introducción	33
B.2. Objetivos generales	33
B.3. Catálogo de requisitos	35
B.4. Especificación de requisitos	37
Apéndice C Especificación de diseño	43
C.1. Introducción	43
C.2. Diseño de datos y capa de persistencia	43
C.3. Diseño procedimental y flujo de ejecución	46
C.4. Diseño arquitectónico y aislamiento de capas	55
C.5. Herramientas de gestión del ciclo de vida, calidad de código y empaquetado	57
Apéndice D Documentación técnica de programación	59
D.1. Introducción	59

D.2. Estructura de directorios	59
D.3. Manual del programador	61
D.4. Pruebas del sistema	63
Apéndice E Documentación de usuario	73
E.1. Introducción	73
E.2. Requisitos de usuario	74
E.3. Instalación	75
E.4. Manual del usuario	78
Apéndice F Sostenibilización curricular	85
F.1. Introducción	85
F.2. Contribución a los Objetivos de Desarrollo Sostenible	85
F.3. Análisis del impacto del proyecto por dimensiones	88
F.4. Reflexión ética y profesional	90
F.5. Conclusión	91
Bibliografía	93

Índice de figuras

A.1. Evolución del trabajo y gráfico burn-down del Sprint 1. Fuente: Elaboración Propia.	4
A.2. Evolución del trabajo y gráfico burn-down del Sprint 2. Fuente: Elaboración Propia.	6
A.3. Evolución del trabajo y gráfico burn-down del Sprint 3. Fuente: Elaboración Propia.	8
A.4. Evolución del trabajo y gráfico burn-down del Sprint 4. Fuente: Elaboración Propia.	11
A.5. Evolución del trabajo y gráfico burn-down del Sprint 5. Fuente: Elaboración Propia.	13
A.6. Evolución del trabajo y gráfico burn-down del Sprint 6. Fuente: Elaboración Propia.	16
A.7. Evolución del trabajo y gráfico burn-down del Sprint 7. Fuente: Elaboración Propia.	19
A.8. Evolución del trabajo y gráfico burn-down del Sprint 8. Fuente: Elaboración Propia.	22
A.9. Evolución del trabajo y gráfico burn-down del Sprint 9. Fuente: Elaboración Propia.	24
A.10. Evolución del trabajo y gráfico burn-down del Sprint 10. Fuente: Elaboración Propia.	27
C.1. Modelo conceptual y diagrama de tablas de la base de datos local. Fuente: Elaboración Propia.	45
C.2. Diagrama Entidad-Relación de la base de datos local. Fuente: Elaboración Propia.	46

C.3. Diagrama de secuencia UML del análisis de una muestra de miel. Se detalla la sincronía de las llamadas, los tiempos de vida de los objetos y la delegación a hilos de trabajo paralelos. Fuente: Elaboración Propia.	53
C.4. Diagrama de flujo de datos (DFD). Identificación de procesos, almacenes de persistencia temporales e interfaces de intercambio de estructuras de datos a través del sistema HIVES. Fuente: Elaboración Propia.	54
D.1. Informe de calidad del código generado por SonarQube en la configuración inicial (sonar.sources=src). El proyecto supera el <i>Quality Gate</i> con calificación A en seguridad, fiabilidad y mantenibilidad; no obstante, la cobertura de pruebas registra 0,0 % por ausencia de informe de cobertura externo, y el índice de duplicaciones alcanza el 45,7 % debido a la presencia simultánea de código legado y código refactorizado en el repositorio. Fuente: Elaboración Propia.	67
D.2. Informe de SonarQube tras la modificación de sonar.sources=src/hives . El incremento no esperado de líneas analizadas (hasta 212.000) y el descenso de Fiabilidad a C evidencian que el analizador indexó dependencias externas incluidas en la ruta configurada, y no únicamente el código fuente del proyecto. Fuente: Elaboración Propia.	69
E.1. Vista general de la ventana principal de la aplicación HIVES. Fuente: Elaboración Propia.	74
E.2. Menú lateral izquierdo de la ventana principal: selector de puerto serie y parámetros de análisis. Fuente: Elaboración Propia.	79
E.3. Ventana principal mostrando los resultados tras completar un análisis de muestra. Fuente: Elaboración Propia.	81
E.4. Ventana de históricos con los filtros de búsqueda disponibles. Fuente: Elaboración Propia.	82
F.1. Los 17 Objetivos de Desarrollo Sostenible de la Agenda 2030 de Naciones Unidas. Fuente: [2]	86
F.2. Objetivos de Desarrollo Sostenible sobre los que el sistema HIVES ejerce una contribución directa. Fuente: Elaboración Propia.	87
F.3. Las tres dimensiones de la sostenibilidad y su intersección con el proyecto HIVES. Fuente: Elaboración Propia.	88

Índice de tablas

A.1. Equivalencia entre <i>Story Points</i> y estimación temporal.	2
A.2. Catálogo de tareas del Sprint 1.	2
A.3. Catálogo de tareas del Sprint 2.	5
A.4. Catálogo de tareas del Sprint 3.	7
A.5. Catálogo de tareas del Sprint 4.	9
A.6. Catálogo de tareas del Sprint 5.	12
A.7. Catálogo de tareas del Sprint 6.	14
A.8. Catálogo de tareas del Sprint 7.	17
A.9. Catálogo de tareas del Sprint 8.	20
A.10. Catálogo de tareas del Sprint 9.	23
A.11. Catálogo de tareas del Sprint 10.	25
A.12. Desglose de horas de desarrollo por sprint.	29
A.13. Desglose del coste de personal incluyendo cargas sociales.	29
A.14. Costes de fabricación del prototipo Hardware	30
A.15. Resumen de costes totales del proyecto HIVES.	30
A.16. Librerías de terceros empleadas, versiones mínimas y licencias. Se distinguen las dependencias de ejecución (<i>runtime</i>) de las empleadas exclusivamente en la fase de pruebas.	31
B.1. Tabla requisitos funcionales	35
B.2. Tabla requisitos no funcionales	36
B.3. CU-01 Analizar Muestra de Miel.	37
B.4. CU-02 Generar y Exportar Reporte PDF.	38
B.5. CU-03 Consultar Histórico de Análisis.	39
C.1. Mapeo de criticidad de escenarios de error y estrategias de miti- gación en el sistema HIVES.	52
E.1. Requisitos mínimos del sistema para la ejecución de HIVES. . .	75

E.2. Mensajes de error de la aplicación y acciones correctoras.	83
F.1. Correspondencia entre los Objetivos de Desarrollo Sostenible y las aportaciones del proyecto HIVES.	89

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Para el desarrollo de este proyecto, se ha optado por seguir la metodología ágil Scrum [22], dado que es el marco de trabajo más extendido en la industria del desarrollo de software y se encuentra en consonancia con la formación adquirida durante los estudios académicos y las prácticas en empresa.

Como herramienta central para la gestión del proyecto, se ha utilizado el software Jira de Atlassian. Esta plataforma permite realizar un seguimiento estructurado de las tareas, monitorizar su estado de ejecución y facilitar una gestión eficiente de los sprints. Los elementos principales empleados durante la planificación en Jira han sido los siguientes:

- **Epics:** Empleadas para agrupar y clasificar las tareas en función del módulo o área del sistema sobre las que tienen impacto.
- **Tasks (Tareas):** Unidades individuales de trabajo planificadas que resultan necesarias para la consecución de los objetivos del proyecto.
- **Vista de Backlog:** Herramienta de visualización utilizada para monitorizar la situación actual, el progreso de todas las tareas pendientes y de los sprints en curso.

Dentro de la herramienta, se ha establecido como norma que cada tarea debe contar con una estimación temporal, un nivel de prioridad y una etiqueta identificativa para facilitar su clasificación. Para estandarizar las

mediciones del esfuerzo requerido en el proyecto, se ha empleado la métrica de *Story Points*, cuya equivalencia temporal se detalla en la siguiente tabla:

Story Points	Estimación
1	1 hora
2	2 horas
3	3 horas
4	4 horas
5	5 horas
8	8 horas
13	13 horas
15	15 horas

Tabla A.1: Equivalencia entre *Story Points* y estimación temporal.

A.2. Planificación temporal

A continuación, se exponen los resúmenes correspondientes a cada sprint, los cuales han sido redactados y documentados tras la finalización y revisión de cada iteración.

Sprint 1: Estructuración Inicial del Proyecto

- **Período:** 26 de marzo - 2 de abril de 2026 (7 días).
- **Objetivo del Sprint:** Estructuración inicial del proyecto, configuración del repositorio de control de versiones y definición de los componentes técnicos fundamentales.
- **Esfuerzo estimado:** 16 horas de desarrollo.

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-8	Documentación inicial	[SW-DOCS-01]	5 pts	5 pts	Completada
SCRUM-10	Organizar carpetas repo	[SW-DOCS-01]	2 pts	2 pts	Completada
SCRUM-11	Definir modelos BD	[SW-ANALYSIS-01]	3 pts	3 pts	Completada
SCRUM-12	Probar interfaz PySide6	[SW-ANALYSIS-01]	4 pts	4 pts	Completada
SCRUM-13	Redactar README	[SW-ANALYSIS-01]	2 pts	2 pts	Completada

Tabla A.2: Catálogo de tareas del Sprint 1.

- **Total estimado inicialmente:** 5 *Story Points* (en el *Sprint Backlog* original).
- **Total real del sprint:** 16 *Story Points*.
- **Tasa de finalización:** 16/16 (100 %).

Dinámica del Sprint

Durante la fase de planificación (*Sprint Planning*), se programaron inicialmente 5 puntos de historia. Pero, tras un proceso de revisión y refinamiento del *Product Backlog* efectuado el 27 de marzo, se identificaron nuevas tareas que resultaban esenciales para asegurar la correcta estructuración inicial del software:

- **SCRUM-10:** Organizar carpetas repo (2 puntos).
- **SCRUM-11:** Definir modelos BD (3 puntos).
- **SCRUM-12:** Probar interfaz PySide6 (4 puntos).
- **SCRUM-13:** Redactar *README* (2 puntos).

Estas tareas fueron incorporadas al *sprint* durante su ejecución. Esta situación no refleja un error de estimación, sino la naturaleza ágil del proyecto, evidenciando una adaptación ante la aparición de nuevas dependencias técnicas.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, mostró el siguiente progreso:

- **27 de marzo:** Inicio del ciclo con 5 puntos planificados.
- **27-28 de marzo:** Incorporación controlada de 11 puntos adicionales al *sprint backlog* (alcanzando un total de 16 puntos de esfuerzo real).
- **29 de marzo - 2 de abril:** Reducción constante y gradual del trabajo pendiente.
- **2 de abril:** Cierre del *sprint* sin tareas pendientes.

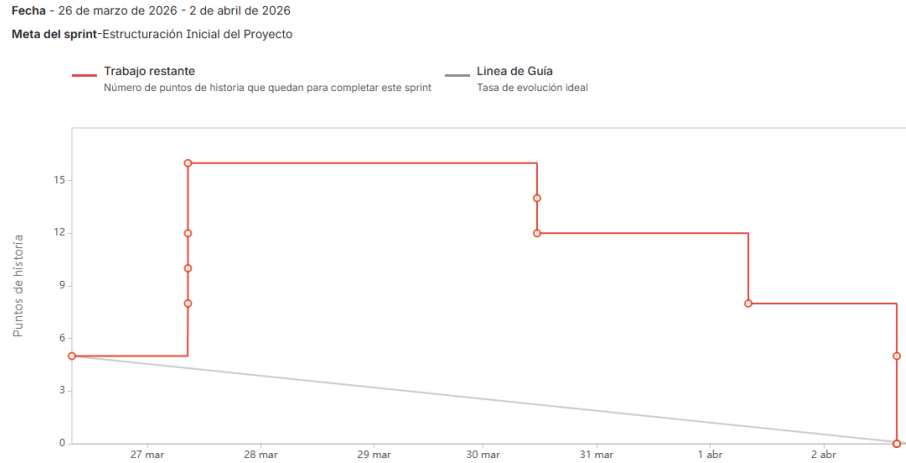


Figura A.1: Evolución del trabajo y gráfico burn-down del Sprint 1. Fuente: Elaboración Propia.

Observaciones clave:

- El incremento de puntos registrado el 28 de marzo fue una acción planificada y justificada, evitando una desviación incontrolada del alcance (*scope creep*).
- Se logró completar el 100 % de las tareas programadas dentro del marco temporal establecido.
- La velocidad de desarrollo alcanzada se estableció en 16 puntos de historia en 7 días.
- No se registraron impedimentos técnicos ni bloqueos funcionales durante la iteración.
- **Eficiencia temporal:** Las tareas se completaron dentro del margen de las 16 horas estimadas, sin incurrir en sobrecostos temporales.

Sprint 2: Siguiendo fase de estructuración

- **Período:** 2 de abril - 9 de abril de 2026 (7 días).
- **Objetivo del Sprint:** Continuar con la formalización inicial del proyecto y empezar a añadir funcionalidades del sensor.
- **Esfuerzo estimado:** 17 horas de desarrollo.

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-14	Pensar funciones Arduino para poner en la interfaz	[SW-HW-01]	5 pts	5 pts	Completada
SCRUM-15	Mockup Interfaz	[SW-DESK-01]	2 pts	2 pts	Completada
SCRUM-16	Documentar sprint	[SW-DOCS-01]	1 pts	1 pts	Completada
SCRUM-17	Comunicación Python Serial	[SW-HW-01]	3 pts	3 pts	Completada
SCRUM-18	Investigación Análisis Polínicos	[SW-ANALYSIS-01]	3 pts	3 pts	Completada
SCRUM-19	Investigar Modelo	[SW-AI-01]	3 pts	3 pts	Completada

Tabla A.3: Catálogo de tareas del Sprint 2.

- **Total estimado inicialmente:** 17 *Story Points* (en el *Sprint Backlog* original).
- **Total real del sprint:** 17 *Story Points*.
- **Tasa de finalización:** 17/17 (100 %).

Dinámica del Sprint

Durante la fase de planificación (*Sprint Planning*), se programaron inicialmente 17 puntos de historia, centrándose en seguir formalizando las distintas decisiones sobre el diseño de la aplicación, así como en empezar a establecer una comunicación entre el dispositivo y la aplicación.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, mostró el siguiente progreso a lo largo de la iteración:

- **3 al 5 de abril:** Inicio del ciclo. La línea de progreso (roja) se mantiene plana en los 17 puntos de historia originales. Esto indica que las tareas se encontraban en fase de desarrollo.
- **6 y 7 de abril:** Se registran los primeros cierres formales de tareas, reduciendo el trabajo pendiente de manera escalonada de 17 a 13 puntos.
- **8 de abril:** Jornada de mayor consolidación e integración, evidenciada por una caída pronunciada en el gráfico que reduce el trabajo pendiente de 13 a 5 puntos de historia.

- **9 de abril:** Cierre y validación de las tareas finales, alcanzando el objetivo de 0 puntos pendientes en el último día del *sprint*.

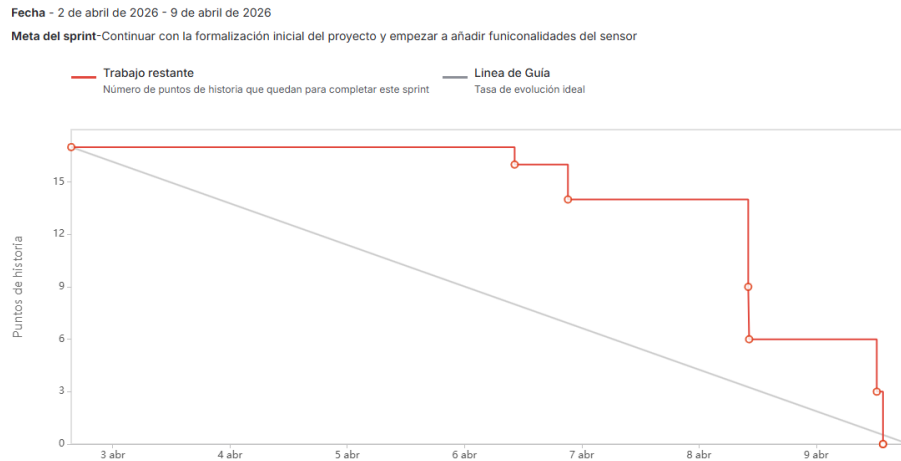


Figura A.2: Evolución del trabajo y gráfico burn-down del Sprint 2. Fuente: Elaboración Propia.

Observaciones clave:

- **Desviación inicial:** Durante la primera mitad de la iteración, la curva real de progreso se situó por encima de la línea ideal de resolución (gris). Esto fue causado porque las primeras tareas realizadas fueron las que requerían mayor tiempo de investigación y estructuración antes de poder considerarse cerradas.
- **Estabilidad del alcance:** No se aprecian picos ascendentes en la línea de trabajo, lo que confirma que no se introdujeron tareas imprevistas a mitad del ciclo.
- Se logró completar el 100 % de las tareas dentro del marco temporal establecido, finalizando la iteración puntualmente el 9 de abril.
- No se registraron impedimentos técnicos ni bloqueos funcionales críticos que comprometieran la entrega.
- **Velocidad validada:** Se ha demostrado la capacidad de absorber y completar una carga de 17 puntos de historia en un ciclo de 7 días, estableciendo la primera métrica real de velocidad (*velocity*) del proyecto.

Sprint 3: Implementación de Interfaz, Persistencia e Integración Física

- **Período:** 10 de abril - 16 de abril de 2026 (7 días).
- **Objetivo del Sprint:** Desarrollar la primera iteración funcional de la interfaz gráfica, establecer la persistencia de datos (base de datos y exportación CSV) y realizar pruebas físicas de medición con el hardware.
- **Esfuerzo estimado:** 25 horas de desarrollo (*Story Points*).

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-20	Documentar Sprint Anterior	[SW-DOCS-01]	2 pts	2 pts	Finalizado
SCRUM-21	Crear Interfaz Gráfica	[SW-DESK-01]	5 pts	5 pts	Finalizado
SCRUM-22	Colocar Datos en la UI	[SW-DESK-01]	5 pts	5 pts	Finalizado
SCRUM-23	Crear BD para los datos	[SW-STORAGE-01]	2 pts	2 pts	Finalizado
SCRUM-24	Pruebas medición con placa LEDs y adaptación de carcasa	[SW-QA-01]	8 pts	8 pts	Finalizado
SCRUM-25	Generador CSV	[SW-REPORT-01]	3 pts	3 pts	Finalizado

Tabla A.4: Catálogo de tareas del Sprint 3.

- **Total estimado inicialmente:** 25 *Story Points* (en el *Sprint Backlog* original).
- **Total real del sprint:** 25 *Story Points*.
- **Tasa de finalización:** 25/25 (100 %).

Dinámica del Sprint

Durante la fase de planificación (*Sprint Planning*), se programaron 25 puntos de historia. El foco de esta iteración evolucionó desde la formalización hacia la implementación técnica real, centrándose principalmente en la interfaz de usuario, la consolidación de la capa de almacenamiento (SQLite y CSV) y la validación del hardware (placa de LEDs y carcasa).

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, mostró el siguiente progreso a lo largo de la iteración:

- **10 al 12 de abril:** Fase de arranque. La línea de progreso (roja) se mantiene en los 25 puntos originales, indicando el inicio del desarrollo simultáneo de las tareas como la interfaz gráfica y las pruebas de hardware.
- **12 y 13 de abril:** Primeros cierres formales, registrando dos caídas consecutivas que reducen el trabajo pendiente de 25 a 21 puntos al cierre de tareas menores como la documentación y la creación de la BD.
- **14 de abril:** Reducción escalonada hasta los 18 puntos, indicando la finalización del módulo de generación de CSV.
- **15 y 16 de abril:** Consolidación final. Se produce el cierre de las tareas más pesadas, la interfaz gráfica y las pruebas de hardware, terminando el gráfico desde los 18 hasta los 0 puntos en la recta final del *sprint*.

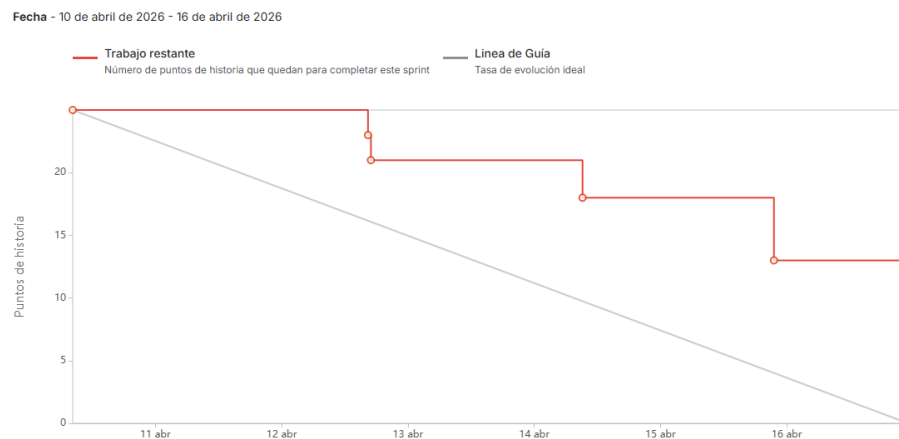


Figura A.3: Evolución del trabajo y gráfico burn-down del Sprint 3. Fuente: Elaboración Propia.

Observaciones clave:

- **Desviación respecto a la línea ideal:** La curva real se mantuvo por encima de la línea ideal (gris) durante casi todo el ciclo. Este patrón, conocido como *Late Drop* (caída tardía), es habitual cuando el *sprint* contiene tareas de gran peso (como los 8 puntos de la tarea relacionada con las pruebas de medición con las placas LED y la adaptación de la carcasa) que no pueden cerrarse hasta estar completamente testadas.

- **Estabilidad del alcance:** La parte superior del gráfico se mantuvo plana, lo que confirma que no hubo inyección de tareas imprevistas durante la iteración.
- **Aumento de velocidad:** Se logró un incremento significativo en la capacidad de entrega, pasando de absorber 16 puntos en el Sprint 1 a completar exitosamente **25 puntos** en este Sprint 3 dentro de la misma franja de 7 días.
- No se reportaron cuellos de botella críticos, logrando una sincronización exitosa entre el desarrollo de software y las pruebas físicas de hardware.

Sprint 4: Inicio del Modelo de Clasificación y Verificación de Interfaz

- **Período:** 17 de abril - 23 de abril de 2026 (7 días).
- **Objetivo del Sprint:** Comenzar con el desarrollo y preparación de los modelos de clasificación, finalizar el prototipo físico para la toma de muestras, y verificar de forma exhaustiva la funcionalidad de la interfaz y la comunicación hardware.
- **Esfuerzo estimado:** 20 horas de desarrollo (*Story Points*).

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-26	Documentar Sprint Anterior	[SW-DOCS-01]	2 pts	2 pts	Finalizado
SCRUM-27	Revisar Datos Interfaz	[SW-QA-01]	1 pts	1 pts	Finalizado
SCRUM-28	Hacer Prueba del modelo	[SW-AI-01]	8 pts	8 pts	Finalizado
SCRUM-29	Comprobar comunicación serial	[SW-QA-01]	1 pts	1 pts	Finalizado
SCRUM-30	Comprobar controles por la interfaz	[SW-QA-01]	1 pts	1 pts	Finalizado
SCRUM-31	Finalizar Diseño Carcasa	[SW-HW-01]	3 pts	3 pts	Finalizado
SCRUM-32	Adaptar control luces con las nuevas placas de los LEDs	[SW-HW-01]	4 pts	4 pts	Finalizado

Tabla A.5: Catálogo de tareas del Sprint 4.

- **Total estimado inicialmente:** 20 *Story Points* (en el *Sprint Backlog* original).
- **Total real del sprint:** 20 *Story Points*.
- **Tasa de finalización:** 20/20 (100 %).

Dinámica del Sprint

Durante la fase de planificación (*Sprint Planning*), se programaron 20 puntos de historia con un enfoque claro en la integración y validación. El esfuerzo se distribuyó en tres frentes: la finalización del hardware (diseño de la carcasa y adaptación de placas LED) para permitir el registro de medidas sobre muestras reales; la verificación minuciosa de la interfaz y la comunicación serie (identificando y corrigiendo posibles fallos en los controles); y la preparación de las primeras pruebas reales del modelo de clasificación.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, mostró el siguiente progreso a lo largo de la iteración:

- **17 al 19 de abril:** Fase de desarrollo e integración. La línea de progreso (roja) se mantiene plana en los 20 puntos originales, indicando un periodo de trabajo activo en modificaciones de hardware y ajustes de interfaz sin cierres formales.
- **20 de abril:** Se registra el primer cierre formal de la iteración (correspondiente a labores de documentación), reduciendo el trabajo pendiente de 20 a 18 puntos de historia.
- **21 y 22 de abril:** El día 22 marca la finalización de los componentes físicos. Se observa una caída pronunciada: primero baja a 15 puntos (cierre del diseño de la carcasa) y posteriormente a 11 puntos (adaptación del control de luces LED).
- **23 de abril:** Consolidación y validación final. En la última jornada se agrupan múltiples cierres consecutivos de tareas de QA (comprobación de controles, datos y comunicación serial), reduciendo el marcador de 11 a 10, luego a 9 y a 8. Finalmente, el cierre de la tarea más compleja (prueba del modelo) desploma el gráfico a 0 puntos.

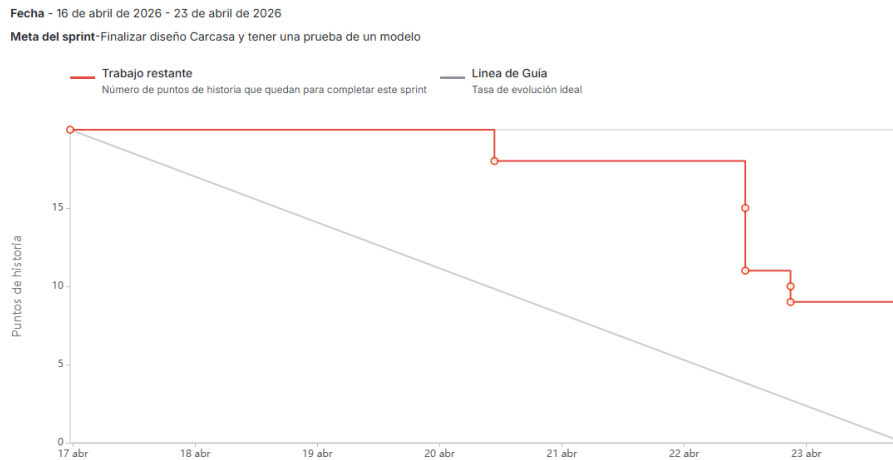


Figura A.4: Evolución del trabajo y gráfico burn-down del Sprint 4. Fuente: Elaboración Propia.

Observaciones clave:

- **Comportamiento en cascada (*Late Drop*):** La gran mayoría del peso del *sprint* se liberó en las últimas 48 horas del ciclo. Este patrón refleja la alta dependencia estructural de este ciclo: no se podían validar los controles de la interfaz ni probar el modelo hasta no tener el hardware (carcasa y LEDs) firmemente adaptado, unido al tiempo empleado en los entrenamientos y la búsqueda de alternativas en la nube para acelerar el proceso.
- **Estabilidad del alcance:** La línea superior de puntos se mantuvo inalterada a lo largo del periodo, confirmando una planificación precisa sin introducción de tareas imprevistas (*scope creep*).
- **Validación integral:** Se logró completar el 100 % de las tareas dentro del marco temporal.

Sprint 5: Integración del Modelo IA y Ampliación de la Interfaz

- **Período:** 24 de abril - 30 de abril de 2026 (7 días).
- **Objetivo del Sprint:** Integrar el modelo de Inteligencia Artificial dentro del flujo de la aplicación, implementar el módulo visual para la

consulta del histórico de análisis, y habilitar la configuración dinámica de parámetros desde la interfaz gráfica.

- **Esfuerzo estimado:** 22 horas de desarrollo (*Story Points*).

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-33	Documentar Sprint Anterior	[SW-DOCS-01]	1 pts	1 pts	Finalizado
SCRUM-35	Incorporar ventana históricos	[SW-DESK-01]	8 pts	8 pts	Finalizado
SCRUM-34	Incorporar Modelo	[SW-ANALYSIS-01]	8 pts	8 pts	Finalizado
SCRUM-36	Permitir cambiar parámetros de análisis desde la interfaz	[SW-DESK-01]	5 pts	5 pts	Finalizado

Tabla A.6: Catálogo de tareas del Sprint 5.

- **Total estimado inicialmente:** 22 *Story Points* (en el *Sprint Backlog* original).
- **Total real del sprint:** 22 *Story Points*.
- **Tasa de finalización:** 22/22 (100 %).

Dinámica del Sprint

Durante la fase de planificación (*Sprint Planning*), se programaron 22 puntos de historia. Esta iteración representó un hito en el acoplamiento de los diferentes subsistemas. El esfuerzo se focalizó en dos áreas principales: por un lado, la incorporación del modelo de clasificación al núcleo de la aplicación, conectando las lecturas del hardware con las predicciones del modelo; por otro lado, la ampliación de la interfaz gráfica añadiendo una ventana de persistencia histórica y permitiendo al usuario inyectar parámetros de análisis de forma dinámica.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, mostró el siguiente progreso a lo largo de la iteración:

- **24 de abril:** Inicio del ciclo. Se registra un cierre temprano asociado a labores de gestión documental, reduciendo el total pendiente de 22 a 21 puntos.

- **25 al 27 de abril:** Fase de desarrollo pesado. La curva (roja) se mantiene en los 21 puntos, denotando un periodo de implementación ininterrumpida sin finalización formal de tareas debido a la alta carga técnica de la interfaz de históricos y el modelo IA (8 puntos respectivamente).
- **28 de abril:** Primer hito de integración visual. Se produce el cierre de la ventana de históricos, marcando un descenso pronunciado en el gráfico desde los 21 hasta los 13 puntos.
- **29 de abril:** Consolidación del núcleo algorítmico y visual. En esta jornada se produce un cierre masivo en dos fases: inicialmente el gráfico cae a 5 puntos (tras la exitosa incorporación del modelo predictivo) y, posteriormente, culmina su descenso a 0 puntos con la validación de la configuración de parámetros dinámicos desde la interfaz.

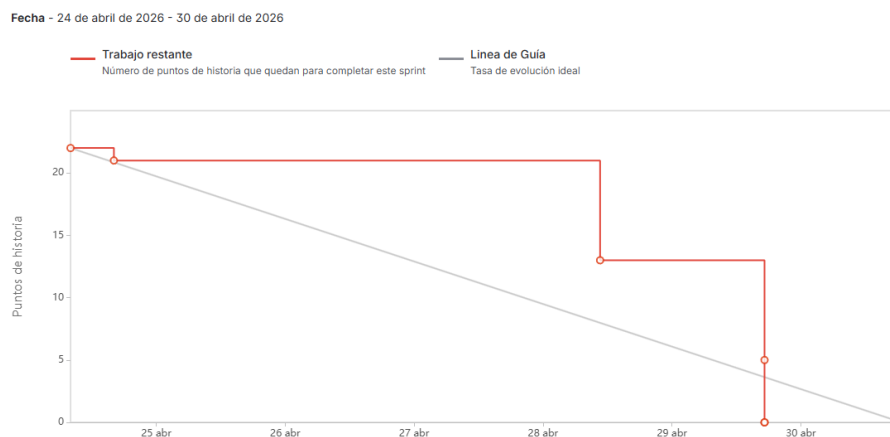


Figura A.5: Evolución del trabajo y gráfico burn-down del Sprint 5. Fuente: Elaboración Propia.

Observaciones clave:

- **Patrón de entrega orientada a valor:** A diferencia de *sprints* anteriores donde había micro-tareas, aquí se han gestionado épicas más grandes (tareas de 8 puntos). La caída brusca en los días 28 y 29 demuestra que las funcionalidades se probaron y validaron como bloques enteros antes de darse por terminadas.
- **Estabilidad y madurez del alcance:** La precisión milimétrica entre la estimación inicial y la finalización (100 % de las tareas cerradas antes

de finalizar el día 29) confirma la madurez adquirida en la estimación del esfuerzo algorítmico y de interfaz gráfica.

- **Rendimiento sostenido:** Se ha logrado estabilizar su velocidad (*velocity*) en la franja de los 20-25 puntos por iteración, demostrando una cadencia de trabajo altamente predecible y eficiente para las fases venideras del proyecto.

Sprint 6: Ajustes de Predicción, Calibración y Replanificación Activa

- **Período:** 1 de mayo - 7 de mayo de 2026 (7 días).
- **Objetivo del Sprint:** Refinar el proceso de entrenamiento del modelo, integrar la lógica predictiva al tomar muestras en tiempo real y garantizar la consistencia visual de los datos históricos. Adicionalmente, adaptar la planificación para incluir funcionalidades de calibración requeridas para la inminente fase de toma de muestras.
- **Esfuerzo estimado original:** 20 horas de desarrollo (*Story Points*).
- **Esfuerzo total gestionado (tras cambios):** 25 *Story Points*.

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-38	Documentar Sprint Anterior	[SW-DOCS-01]	1 pts	1 pts	Finalizado
SCRUM-37	Verificar consistencia datos y vista de datos	[SW-QA-01]	2 pts	2 pts	Finalizado
SCRUM-39	Acabar entrenamientos y probar el funcionamiento del modelo	[SW-AI-01]	5 pts	5 pts	Finalizado
SCRUM-42	Añadir lógica predicción al tomar una muestra	[SW-DESK-01]	5 pts	5 pts	Finalizado
SCRUM-40	Revisar memoria y actualizar cambios	[SW-DOCS-01]	2 pts	2 pts	Finalizado
SCRUM-41	Añadir botón predecir muestras pasadas	[SW-DESK-01]	5 pts	5 pts	Bloqueada
SCRUM-43	Añadir botón y lógica calibración (Inyectada)	[SW-HW-01]	-	5 pts	Finalizado

Tabla A.7: Catálogo de tareas del Sprint 6.

- **Total completado:** 20 *Story Points* (excluyendo la tarea bloqueada).

- **Tasa de finalización efectiva:** 80 % (20/25 puntos totales gestionados en el tablero). *El 100 % de las tareas planificadas fueron completadas; el 80 % corresponde al cómputo incluyendo la tarea bloqueada.*

Dinámica del Sprint y Replanificación Activa

Durante la *Sprint Planning*, se programaron inicialmente 20 puntos. Sin embargo, la ejecución del ciclo requirió una adaptación táctica del *Product Backlog*.

Por un lado, la tarea **SCRUM-41** (5 pts) quedó en estado **Bloqueada**. Esto fue debido a dependencias técnicas no resueltas en la arquitectura de datos subyacente, impidiendo su correcta integración en este ciclo. Por otro lado, ante la inminencia del siguiente *sprint* (destinado a la captura de muestras reales), se identificó como crítica la necesidad de calibrar el sensor de forma interactiva. Por ello, en una clara demostración de agilidad, se **inyectó anticipadamente la tarea SCRUM-43** (5 pts) al *sprint backlog*, compensando el esfuerzo de la tarea bloqueada y garantizando la viabilidad de la siguiente fase del proyecto.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, ilustra visualmente estos cambios de alcance:

- **1 y 2 de mayo:** Configuración del tablero. El gráfico muestra una carga inicial masiva de tareas el día 2, elevando la línea de trabajo pendiente bruscamente hasta los 20 puntos de historia.
- **3 de mayo:** Primer cierre. Se completa la documentación del *sprint* anterior (SCRUM-38), reduciendo el trabajo pendiente a 19 puntos.
- **4 y 5 de mayo:** Ejecución y validación. El día 5 se produce una caída significativa del gráfico (de 19 a 12, y luego a 7 puntos). Esto refleja el cierre simultáneo de la verificación de datos (SCRUM-37), la lógica de predicción en tiempo real (SCRUM-42) y las pruebas de entrenamiento del modelo (SCRUM-39).
- **6 de mayo (Punto de inflexión):** Se observa un **pico ascendente** en el gráfico, pasando de 7 a 12 puntos de trabajo pendiente. Esto refleja el momento exacto en el que se inyecta al *sprint* la tarea imprevista de calibración (SCRUM-43). Ese mismo día, la tarea es desarrollada y cerrada, devolviendo la gráfica a los 7 puntos originales.

- **7 de mayo:** Cierre administrativo. Se finaliza la revisión de la memoria (SCRUM-40), reduciendo el gráfico a 5 puntos pendientes. Estos 5 puntos finales corresponden a la tarea SCRUM-41, la cual finaliza el ciclo en estado bloqueado y pasará al *backlog* de futuras iteraciones.

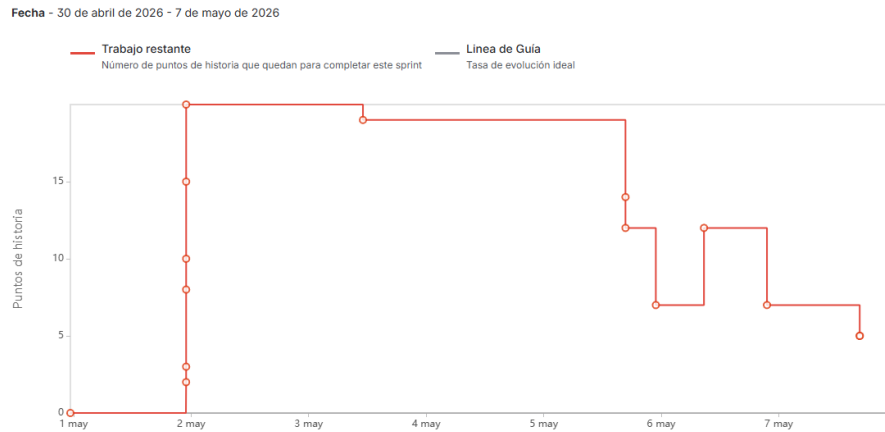


Figura A.6: Evolución del trabajo y gráfico burn-down del Sprint 6. Fuente: Elaboración Propia.

Observaciones clave:

- **Gestión del *Scope Creep* y adaptación:** El pico ascendente del 6 de mayo evidencia la flexibilidad del marco ágil. Lejos de ser un fallo de planificación, la inyección de la tarea de calibración fue una decisión estratégica para desatascar las dependencias físicas del hardware de cara al siguiente *sprint*.
- **Transparencia en el bloqueo:** La retención de la tarea SCRUM-41 demuestra un control de calidad riguroso, evitando integrar código no funcional (*technical debt*) en la rama principal (*main*).
- **Mantenimiento de la velocidad:** A pesar de las fricciones y el cambio de contexto (*context switching*), se logró completar **20 puntos de historia reales**, manteniendo la línea de velocidad establecida en ciclos anteriores y asegurando la preparación del sistema para la toma de muestras empíricas.

Sprint 7: Toma de Muestras Empíricas, Refactorización y Ajustes

- **Período:** 7 de mayo - 14 de mayo de 2026 (7 días).
- **Objetivo del Sprint:** Realizar la toma de muestras físicas en entorno real, integrar los resultados de calibración en la lógica de análisis, refactorizar el código base para mejorar su mantenibilidad y realizar ajustes finos en la interfaz de usuario.
- **Esfuerzo total gestionado:** 26 horas de desarrollo (*Story Points*), incluyendo el remanente bloqueado del ciclo anterior.

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-44	Documentar Sprint Anterior	[SW-DOCS-01]	1 pts	1 pts	Finalizado
SCRUM-45	Reorganizar el código	[SW-CORE-01]	8 pts	8 pts	Finalizado
SCRUM-46	Ajustar con resultado de calibraciones	[SW-AI-01]	4 pts	4 pts	Finalizado
SCRUM-47	Revisar lógica análisis y mostrado de datos	[SW-QA-01]	5 pts	5 pts	Finalizado
SCRUM-48	Ajustar tamaños interfaz	[SW-DESK-01]	3 pts	3 pts	Finalizado
SCRUM-41	Añadir botón predecir muestras pasadas	[SW-DESK-01]	5 pts	5 pts	Bloqueada

Tabla A.8: Catálogo de tareas del Sprint 7.

- **Total completado:** 21 *Story Points*.
- **Tasa de finalización efectiva:** 100 % de las nuevas tareas planificadas (21/21 pts). La tarea heredada SCRUM-41 mantiene su estado bloqueado (5 pts pendientes).

Dinámica del Sprint y Replanificación por Trabajo de Campo

Este *sprint* presentó una dinámica atípica debido a la interacción directa con el entorno físico. Los primeros días de la iteración se dedicaron de forma exclusiva al trabajo de campo: la toma de medidas físicas con el prototipo sobre muestras reales de miel. Esta fase empírica generó una latencia natural en la definición técnica de los siguientes pasos. Una vez recolectados y analizados los datos iniciales, se definieron e inyectaron las nuevas tareas al *Product Backlog*, priorizando la integración de las calibraciones y una profunda refactorización del código (8 puntos) para preparar el sistema de cara a su versión final.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, ilustra de forma clara el impacto del trabajo de campo en la planificación técnica:

- **7 al 10 de mayo:** Fase de trabajo empírico. El gráfico arranca mostrando únicamente los 5 puntos heredados de la tarea bloqueada (SCRUM-41). Durante estos días no se registra actividad de desarrollo de software, ya que el esfuerzo estaba concentrado en la captura física de muestras.
- **11 de mayo (Pico de inyección):** Tras analizar los resultados de las muestras, se definen las necesidades técnicas y se inyectan 21 nuevos puntos de historia al tablero, provocando un salto vertical abrupto en la gráfica hasta alcanzar los 26 puntos totales. Ese mismo día se produce el primer cierre (documentación, bajando a 25 puntos).
- **12 de mayo:** Consolidación analítica. Se cierran simultáneamente las tareas de ajuste de calibraciones (4 pts) y la revisión de la lógica de análisis (5 pts), provocando una caída pronunciada de 9 puntos que sitúa el trabajo pendiente en 16.
- **13 de mayo:** Hito de arquitectura. Se finaliza la tarea de mayor peso del *sprint* (Reorganizar el código, 8 pts), reduciendo el gráfico de 16 a 8 puntos.
- **14 de mayo:** Cierre visual. Se completa el ajuste de los tamaños de la interfaz gráfica (3 pts). El *sprint* finaliza con la curva estabilizada en 5 puntos, correspondientes a la tarea SCRUM-41 que permanece a la espera de desbloqueo.



Figura A.7: Evolución del trabajo y gráfico burn-down del Sprint 7. Fuente: Elaboración Propia.

Observaciones clave:

- **Flexibilidad metodológica:** El gráfico ilustra un escenario real de adaptación ágil. La inyección masiva de tareas en el día 11 no representa un fallo de planificación, sino una respuesta técnica directa a los descubrimientos realizados durante los dos días de trabajo físico de campo.
- **Deuda técnica y refactorización:** La asignación de 8 puntos a la “Reorganización del código” demuestra madurez en la gestión del ciclo de vida del software. Tras la rápida iteración de los *sprints* anteriores para conseguir un prototipo funcional, se priorizó reducir la deuda técnica antes de escalar el sistema.
- **Alta velocidad de resolución:** A pesar del retraso inicial en la definición de tareas, se demostró una gran eficiencia al desarrollar, probar y cerrar 21 puntos de historia en un marco temporal comprimido de apenas 4 días (del 11 al 14 de mayo).

Sprint 8: Consolidación Documental y Pruebas de Despliegue

- **Período:** 15 de mayo - 22 de mayo de 2026 (7 días).

- **Objetivo del Sprint:** Volcar los descubrimientos y desarrollos técnicos en la memoria del proyecto (documentación de avances), alinear la redacción con los resultados de las pruebas empíricas, e iniciar la fase de empaquetado del software para su distribución como ejecutable independiente.
- **Esfuerzo total gestionado:** 26 horas de desarrollo (*Story Points*), incluyendo tareas redefinidas de ciclos anteriores.

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-51	Documentar sprint anterior	[SW-DOCS-01]	1 pts	1 pts	Finalizado
SCRUM-49	Documentar Avances	[SW-DOCS-01]	15 pts	15 pts	Finalizado
SCRUM-50	Ajustar avances con las pruebas	[SW-QA-01]	5 pts	5 pts	Finalizado
SCRUM-41	Hacer ejecutable y probar compatibilidad	[SW-DEPLOY-01]	5 pts	5 pts	Bloqueada

Tabla A.9: Catálogo de tareas del Sprint 8.

- **Total completado:** 21 *Story Points*.
- **Tasa de finalización efectiva:** 80 % (21/26 pts). La tarea SCRUM-41 finaliza el ciclo en estado bloqueado debido a incidencias de compatibilidad.

Dinámica del Sprint y Evolución de Tareas

Durante la planificación de este ciclo, el enfoque cambió drásticamente desde la implementación de código hacia la consolidación académica y documental. Se asignó una épica de gran tamaño (15 puntos) a la redacción y estructuración de los avances en la memoria del TFG.

La tarea **SCRUM-41** fue redefinida (*re-scoped*) hacia la creación del ejecutable final y la prueba de compatibilidad multiplataforma. Sin embargo, finalmente la tarea resultó **bloqueada** ante la falta del desarrollo de las exportaciones de los análisis a **PDF**. Unido a ello, el empaquetado de aplicaciones Python que integran ecosistemas complejos (como los binarios de TensorFlow para la IA y las librerías dinámicas de PyQt6 para la interfaz) suele generar conflictos de dependencias o rutas relativas al compilarse en un único ejecutable, requiriendo una investigación más profunda que excedía el marco temporal de este *sprint*.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, muestra un patrón extremo de *Late Drop* característico de las tareas de documentación:

- **15 y 16 de mayo:** Inicio del ciclo con 26 puntos. El día 16 se produce una pequeña caída de 1 punto correspondiente al cierre de la documentación rutinaria del *sprint* anterior (SCRUM-51), dejando el marcador en 25 puntos.
- **17 al 20 de mayo:** Valle de desarrollo continuo. La línea de progreso (roja) se mantiene completamente horizontal en los 25 puntos durante cuatro días consecutivos. Este comportamiento es el reflejo directo de estar trabajando en una tarea monolítica de 15 puntos ("Documentar Avances") que no puede fragmentarse y no se considera terminada hasta que la revisión completa del documento está lista.
- **21 de mayo (Caída masiva):** Consolidación del trabajo. En una misma jornada se da por finalizada y validada la redacción de los avances (15 pts) y su ajuste y coherencia con las pruebas realizadas (5 pts). Esto provoca un desplome vertical en el gráfico de 20 puntos en un solo día, situando la curva en la marca de los 5 puntos.
- **22 de mayo:** Cierre del *sprint*. El ciclo finaliza sin más descensos. La línea roja se estabiliza en 5 puntos, los cuales corresponden a la tarea de empaquetado (SCRUM-41) que queda bloqueada a la espera de resolución.

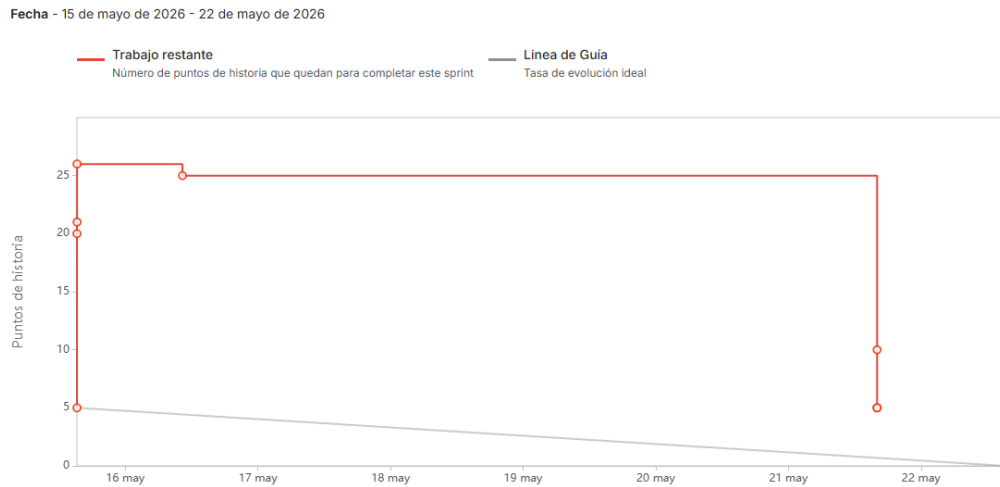


Figura A.8: Evolución del trabajo y gráfico burn-down del Sprint 8. Fuente: Elaboración Propia.

Observaciones clave:

- **Gestión de tareas monolíticas:** La gráfica ilustra perfectamente el impacto de incluir tareas de muy alta puntuación (15 pts) en un *sprint* corto. Aunque la metodología ágil sugiere subdividir estas historias para ver una progresión más escalonada, la naturaleza indivisible de la redacción de un capítulo de memoria justifica esta alteración en la curva ideal.
- **Identificación temprana de riesgos:** El bloqueo de la tarea de despliegue y compatibilidad sirve como alerta temprana (*early warning*). Al haber intentado el empaquetado en el Sprint 8 y no al final del proyecto, se cuenta con margen de maniobra suficiente para investigar y solucionar los conflictos del empaquetador (ej. PyInstaller) sin poner en riesgo la entrega final.

Sprint 9: Calidad de Código, Informes y Despliegue Final

- **Período:** 21 de mayo - 28 de mayo de 2026 (7 días).
- **Objetivo del Sprint:** Implementar la lógica de generación de informes de resultados, asegurar la calidad del código mediante análisis estático

(SonarQube) y, finalmente, resolver las incidencias de empaquetado para generar el ejecutable de la aplicación.

- **Esfuerzo total gestionado:** 16 horas de desarrollo (*Story Points*), incluyendo la resolución de la tarea bloqueada en el ciclo previo.

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-54	Documentar Sprint anterior	[SW-DOCS-01]	1 pts	1 pts	Finalizado
SCRUM-52	Hacer lógica generación de informes	[SW-REPORT-01]	5 pts	5 pts	Finalizado
SCRUM-53	Verificar código SonarQube	[SW-QA-01]	5 pts	5 pts	Finalizado
SCRUM-41	Hacer ejecutable y probar compatibilidad (Desbloqueada)	[SW-DEPLOY-01]	5 pts	5 pts	Finalizado

Tabla A.10: Catálogo de tareas del Sprint 9.

- **Total completado:** 16 *Story Points*.
- **Tasa de finalización efectiva:** 100 % (16/16 pts). Se logró desbloquear y completar la tarea de compatibilidad, cerrando el ciclo sin tareas pendientes.

Dinámica del Sprint y Refinamiento del Alcance

Este ciclo destaca por una fase de planificación atípica. Inicialmente, el *sprint* arrancó únicamente con la tarea heredada (SCRUM-41), lo que generó un desajuste temporal en el tablero. A lo largo de la primera jornada, tras una sesión de refinamiento, se inyectaron progresivamente las tareas restantes (generación de informes y auditoría con SonarQube) hasta conformar el alcance real de 16 puntos. Durante la ejecución, el esfuerzo técnico se dividió equitativamente entre la generación de valor funcional (informes), el aseguramiento de la calidad (análisis de código) y la superación del escollo técnico del empaquetador multiplataforma.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, ilustra perfectamente la inserción progresiva de tareas y la posterior estabilización del ritmo de desarrollo:

- **21 de mayo (Día de planificación activa):** El gráfico inicia mostrando un comportamiento vertical ascendente inusual. Arranca en 5

puntos (tarea heredada) y sufre inyecciones consecutivas a medida que se añaden las nuevas historias al tablero, escalando hasta los 16 puntos totales. Este arranque escalonado es el causante de que la "línea de guía" (gris) adquiera una pendiente más suave de lo habitual.

- **22 de mayo:** Se registra la primera caída de 1 punto (de 16 a 15) tras finalizar la labor documental del *sprint* anterior (SCRUM-54).
- **24 de mayo:** Cierre funcional. Se finaliza la lógica de generación de informes (SCRUM-52), provocando un descenso de 5 puntos que sitúa la curva en 10.
- **25 de mayo:** Hito de calidad. La superación de las métricas de SonarQube (SCRUM-53) resta otros 5 puntos de historia, dejando el gráfico en 5 puntos a mitad del ciclo.
- **26 y 27 de mayo:** Periodo de investigación y pruebas exhaustivas enfocadas exclusivamente en resolver las dependencias del ejecutable.
- **28 de mayo:** Cierre definitivo. En la última jornada se logra empaquetar la aplicación con éxito (SCRUM-41), desplomando la gráfica a 0 puntos y cerrando el ciclo de forma óptima.

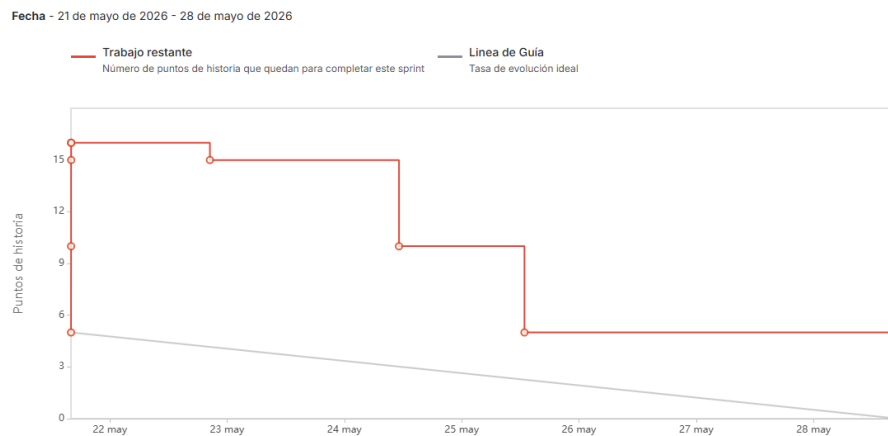


Figura A.9: Evolución del trabajo y gráfico burn-down del Sprint 9. Fuente: Elaboración Propia.

Observaciones clave:

- **Justificación de la desviación de la línea ideal:** El desajuste visual entre la línea roja de trabajo real y la línea gris de tendencia ideal no representa un retraso en la entrega, sino un artefacto visual provocado por la inserción tardía de las estimaciones en el sistema de gestión (Jira) durante el primer día.
- **Resolución de deuda técnica:** El desbloqueo y finalización de la tarea SCRUM-41 valida la decisión tomada en el *sprint* anterior de posponerla. El tiempo invertido en este ciclo permitió aislar las incompatibilidades de las librerías gráficas y de IA, logrando un ejecutable estable.
- **Aseguramiento de Calidad (QA):** La inclusión formal de una tarea de revisión con SonarQube (5 pts) demuestra un enfoque profesional hacia la mantenibilidad, garantizando que el código entregado cumple con los estándares académicos y de la industria antes de su presentación.

Sprint 10: Cierre de Proyecto, Revisión y Entregables Finales

- **Período:** 28 de mayo - 4 de junio de 2026 (7 días).
- **Objetivo del Sprint:** Consolidar la versión definitiva de la aplicación (*build*), finalizar la redacción de la memoria académica y preparar todos los entregables anexos, garantizando un margen de tiempo para la revisión exhaustiva antes de la entrega del TFG programada para el 9 de junio.
- **Esfuerzo estimado:** 21 horas de desarrollo (*Story Points*).

Catálogo de Tareas

ID Tarea	Descripción	Epic	Est. Inicial	Est. Final	Estado
SCRUM-56	Documentar Sprint Anterior	[SW-DOCS-01]	1 pts	1 pts	Finalizado
SCRUM-55	Finalizar Build	[SW-DEPLOY-01]	3 pts	3 pts	Finalizado
SCRUM-57	Finalizar Documentos	[SW-DOCS-01]	13 pts	13 pts	Finalizado
SCRUM-58	Preparar Entregables	[SW-DOCS-01]	4 pts	4 pts	Finalizado

Tabla A.11: Catálogo de tareas del Sprint 10.

- **Total completado:** 21 *Story Points*.
- **Tasa de finalización efectiva:** 100 % (21/21 pts). Todas las tareas se completaron exitosamente antes del fin del ciclo.

Dinámica del Sprint y Estrategia de Cierre

Durante la planificación de este último ciclo, se programaron 21 puntos de historia con una estrategia de ejecución comprimida. Conscientes de la importancia de la fase de validación, se priorizó finalizar la compilación definitiva del software (*build*) y la carga documental más pesada (la memoria final, valorada en 13 puntos) de forma anticipada. Esta compresión temporal se diseñó intencionadamente para crear un periodo de verificación al final del *sprint*, permitiendo una revisión global, lectura de correcciones y auditoría de los entregables sin la presión de la fecha límite.

Análisis del gráfico Burn-Down

La evolución del trabajo, reflejada en el diagrama *burn-down*, muestra el comportamiento típico de un cierre de proyecto o *Release Sprint*:

- **28 y 29 de mayo:** Inicio del ciclo en 21 puntos. El día 29 se registra una ligera caída a 20 puntos, correspondiente al cierre de las labores rutinarias de documentación del ciclo previo (SCRUM-56).
- **30 de mayo al 1 de junio:** Fase de revisión y compilación. La curva (roja) se mantiene plana en los 20 puntos. Durante estos días se llevó a cabo la redacción profunda de la memoria y la generación de los ejecutables finales, tareas que, por su naturaleza, no admitían cierres parciales.
- **2 de junio (Día de consolidación):** Se produce un desplome vertical masivo en el gráfico. En una misma jornada se da por validada la *build* final (3 pts), se aprueba la versión definitiva de los documentos (13 pts) y se empaquetan los entregables (4 pts). El gráfico cae de 20 a 0 puntos de forma inmediata.
- **3 y 4 de junio:** El *sprint* finaliza formalmente con 0 puntos pendientes. Estos días se utilizaron como margen de revisión y preparación, cumpliendo con la estrategia de planificación inicial.

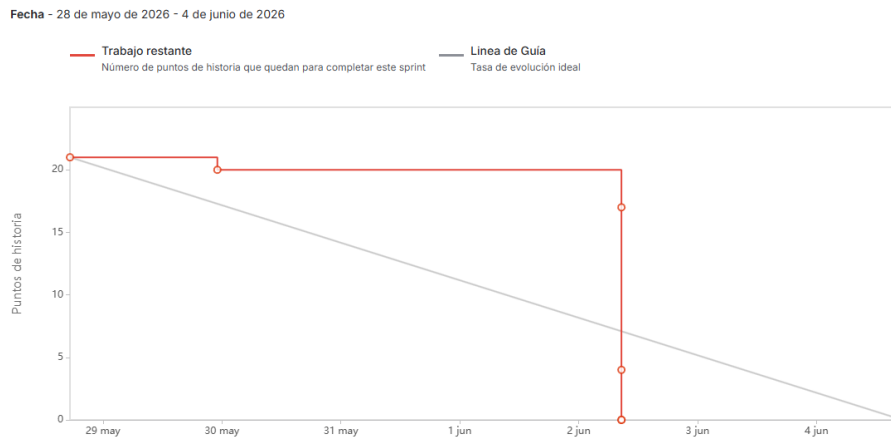


Figura A.10: Evolución del trabajo y gráfico burn-down del Sprint 10. Fuente: Elaboración Propia.

Observaciones clave:

- **Cierre en bloque (*Late Drop* extremo):** La caída en picado del 2 de junio es indicativa de una fase de Control de Calidad (QA) global. En la recta final del proyecto, la compilación y la memoria no pueden cerrarse de forma aislada, ya que cualquier modificación de última hora en el código requiere actualizar la documentación asociada.
- **Cumplimiento holístico:** Se completó el 100% del alcance con antelación, validando la eficacia de la metodología ágil adoptada. La velocidad constante mantenida durante los *sprints* anteriores permitió afrontar esta última semana sin arrastrar deuda técnica, garantizando la calidad final del producto HIVES.

A.3. Estudio de viabilidad

Para confirmar la factibilidad del proyecto, se ha llevado a cabo un análisis estructurado que evalúa diversas áreas clave:

Viabilidad técnica

Para la correcta ejecución del desarrollo, se ha configurado un repositorio de código compartido con el equipo de tutorización. Esta medida permite mantener un control de versiones centralizado y preciso, garantizando el registro histórico y la trazabilidad de todos los cambios implementados.

Adicionalmente, se ha integrado la herramienta SonarQube con el propósito de evaluar de forma continua y asegurar la calidad del código fuente desarrollado.

En lo relativo al entrenamiento del modelo de red neuronal, las fases de experimentación y ajuste de hiperparámetros se han llevado a cabo mediante la plataforma **Google Colaboratory Pro** (Colab Pro). Esta herramienta proporciona acceso bajo demanda a unidades de procesamiento gráfico (GPU) de alto rendimiento en la nube, lo que ha permitido reducir significativamente los tiempos de entrenamiento y ejecutar un elevado número de iteraciones experimentales en un período reducido. La dependencia de esta plataforma queda restringida exclusivamente a la fase de entrenamiento; la inferencia del modelo en producción se ejecuta de forma íntegramente local en la estación de trabajo del usuario, en cumplimiento con el requisito no funcional RNF-04.

Viabilidad temporal

Tal y como se ha expuesto anteriormente, el seguimiento de la viabilidad temporal del proyecto se ha gestionado mediante la herramienta Jira. Esto ha posibilitado una monitorización rigurosa tanto del flujo de tareas como del tiempo efectivo invertido en el desarrollo de cada una de ellas.

Viabilidad económica

El análisis económico del proyecto se fundamenta en la cuantificación de las horas dedicadas a su desarrollo. A partir del registro de Story Points de cada sprint, se obtiene una dedicación total de **199 horas** de desarrollo, tal y como se detalla a continuación:

Iteración	Horas (Story Points)
Sprint 1	16
Sprint 2	17
Sprint 3	25
Sprint 4	20
Sprint 5	22
Sprint 6	20
Sprint 7	21
Sprint 8	21
Sprint 9	16
Sprint 10	21
Total	199

Tabla A.12: Desglose de horas de desarrollo por sprint.

Tomando como referencia las condiciones salariales del mercado actual para un Ingeniero Informático en España en 2026, el salario estimado de un programador junior [12] se sitúa en torno a los **20.436,01 € brutos anuales**. Esta cifra equivale a un coste aproximado de **11,35 €/hora** de trabajo, contando con unas 1.800 horas de trabajo anuales, lo que supone un **coste directo de desarrollo estimado en 2.258,65 €**.

No obstante, el coste real para la empresa no se limita al salario bruto del trabajador. A este importe deben añadirse las **cotizaciones a la Seguridad Social** a cargo del empleador [14], que en España ascienden aproximadamente a un **33 %** sobre el salario bruto, desglosadas principalmente en contingencias comunes (23,60 %), desempleo (5,50 %), formación profesional (0,60 %) y Fondo de Garantía Salarial (0,20 %), entre otras. Aplicando dicho porcentaje sobre el coste de desarrollo calculado, el **coste total de personal a cargo de la empresa asciende a 2.928,53 €**, tal y como se refleja en la Tabla A.13.

Concepto	Importe (€)
Salario bruto (199 h $\times$ 11,35 €/h)	2.258,65
Seguridad Social a cargo del empleador (33 %)	745,35
Coste total de personal	3.004,00

Tabla A.13: Desglose del coste de personal incluyendo cargas sociales.

Adicionalmente, el proyecto ha incurrido en un coste operativo derivado del uso de la plataforma **Google Colaboratory Pro** durante los aproximadamente dos meses que abarcó el ciclo completo de entrenamiento y experimentación con los modelos de aprendizaje automático. A un precio de 10,99 €/mes, el coste total de esta herramienta asciende a **21,98 €**.

Por otro lado, el desarrollo del sistema ha requerido la fabricación de un **prototipo hardware** basado en el sensor espectral AS7265x. Los materiales adquiridos para su construcción se recogen en la Tabla A.14.

Pieza	Cantidad	Precio unitario (€)
Sensor AS7265x [24, 1]	1 sensor	59,97
Placas fabricadas (pack 5)	1 pack	125,00
Arduino	1 placa	18,73
Plástico (carcasa)	1 kg para las distintas pruebas	18,00
Total prototipo		221,70

Tabla A.14: Costes de fabricación del prototipo Hardware

El coste global estimado del proyecto, integrando los costes de personal con cargas sociales, los materiales del prototipo y los costes operativos de las herramientas empleadas, queda recogido en la Tabla A.15.

Concepto	Coste estimado (€)
Coste de personal (salario bruto + Seg. Social)	3.004,00
Google Colaboratory Pro (2 meses)	21,98
Prototipo hardware (sensor, placas, Arduino, etc.)	221,70
Coste total del proyecto	3.247,68

Tabla A.15: Resumen de costes totales del proyecto HIVES.

Viabilidad legal

La viabilidad legal de este proyecto se centra primordialmente en la protección de la propiedad intelectual. La titularidad de los derechos derivados de este Trabajo de Fin de Grado recae sobre el estudiante autor del mismo, de manera conjunta con los tutores académicos, Carlos Cambra y Rubén Ruiz. Al no estar el software distribuido públicamente bajo una licencia formal en el momento de la entrega, los derechos de uso y distribución se rigen por la normativa interna de la Universidad de Burgos aplicable a los trabajos académicos.

En términos de dependencias de software de terceros, el proyecto hace uso de las siguientes librerías externas, cuyas licencias son compatibles con el uso académico y no imponen restricciones sobre el código desarrollado:

Librería	Versión mínima	Licencia
<i>Dependencias de ejecución (runtime)</i>		
Python	3.10	PSF License (permisiva)
PyQt6	6.4	GPL v3 / Licencia comercial
TensorFlow	2.10	Apache License 2.0
PySerial	3.5	BSD 3-Clause
NumPy	1.23	BSD 3-Clause
matplotlib	3.6	PSF-based (matplotlib License)
FPDF2	2.7	LGPL v3
pyinstaller	6.20	GPL 2.0 / Apache License 2
SQLite3	(stdlib)	Dominio público
<i>Dependencias de pruebas (solo desarrollo)</i>		
pytest	7.0	MIT
pytest-qt	4.2	MIT
pytest-mock	3.10	MIT

Tabla A.16: Librerías de terceros empleadas, versiones mínimas y licencias. Se distinguen las dependencias de ejecución (*runtime*) de las empleadas exclusivamente en la fase de pruebas.

Todas las librerías empleadas permiten el uso y estudio del software en contextos académicos. No obstante, cabe destacar que **PyQt6**, el *framework* empleado para la construcción de la interfaz gráfica, se distribuye bajo una **licencia dual**: la versión de código abierto se rige por la **GPL v3**, la cual exigiría que cualquier distribución pública del software que la integre adopte también dicha licencia. Para un uso académico interno y sin distribución comercial, como es el caso de este Trabajo de Fin de Grado, esta condición no supone una restricción operativa. En caso de que el proyecto evolucionara hacia una distribución pública o comercial, se debería evaluar la adquisición de una licencia comercial de PyQt6 por parte de Riverbank Computing [20], o bien la migración a **PySide6** (LGPL v3), que ofrece una API equivalente con menores restricciones de distribución. Cabe destacar que, aunque su instalación no es un requisito para la ejecución de la interfaz final del usuario, durante la fase de experimentación en la nube con Google Colab Pro también se emplearon bibliotecas como XGBoost y CuML[19]. Si bien inicialmente se utilizaron para evaluar y comparar el rendimiento de distintos algoritmos, XGBoost terminó integrándose como la primera etapa de un modelo predictivo mixto (arquitectura híbrida). En este diseño final, el procesamiento inicial realizado por XGBoost genera las bases que alimentan directamente a una red neuronal densa implementada con TensorFlow, conformando así la solución algorítmica definitiva del sistema.

Apéndice B

Especificación de Requisitos

B.1. Introducción

El presente apéndice recoge la especificación de requisitos del sistema HIVES (Honey Identification and Verification by Spectroscopy), una aplicación software desarrollada con el propósito de optimizar y reducir los costes asociados al análisis de muestras de miel, mediante la aplicación de técnicas de Inteligencia Artificial sobre datos espectrométricos.

Este documento describe los objetivos generales del sistema, presenta el catálogo de requisitos funcionales y no funcionales, y proporciona la especificación detallada de los casos de uso principales que definen la interacción entre el usuario y la aplicación.

Esta especificación constituye la base técnica sobre la cual se fundamentan las fases posteriores del ciclo de vida del software, incluyendo el diseño arquitectónico (Apéndice C), la implementación (Apéndice D) y la elaboración de la documentación de usuario (Apéndice E).

B.2. Objetivos generales

El sistema HIVES tiene como finalidad principal proporcionar una solución software accesible y económica para el análisis automatizado de muestras de miel mediante técnicas de espectrometría e Inteligencia Artificial. Los objetivos generales definidos para el sistema son los siguientes:

- OBJ-1: Automatización del análisis. Permitir la captura automática de datos de un espectrómetro mediante conexión por puerto serie con

el dispositivo hardware, eliminando la necesidad de procesamiento manual por parte del operario.

- OBJ-2: Clasificación inteligente mediante IA. Utilizar un modelo de red neuronal implementado con *TensorFlow* para clasificar las muestras de miel en función de sus características espectrales, proporcionando resultados precisos y fiables en un tiempo reducido.
- OBJ-3: Generación de informes técnicos. Producir reportes automatizados en formato PDF que incluyan la fecha del análisis, los resultados de la clasificación, las métricas capturadas y las métricas de evaluación del modelo de IA (niveles de confianza, probabilidades, etc.).
- OBJ-4: Almacenamiento de históricos. Registrar y preservar los datos de los análisis realizados en una base de datos local para posibilitar consultas posteriores, la aplicación de filtros de búsqueda y facilitar futuros procesos de reentrenamiento del modelo predictivo.
- OBJ-5: Usabilidad y accesibilidad. Diseñar e implementar una interfaz gráfica de usuario (GUI) intuitiva, provista de controles descriptivos y flujos de interacción claros, que habilite el uso del sistema a personal sin conocimientos técnicos avanzados.

B.3. Catálogo de requisitos

Requisitos Funcionales

ID	Descripción	Prioridad
RF-01	El sistema debe permitir al usuario iniciar un nuevo análisis de una muestra de miel.	Alta
RF-02	El sistema debe establecer una conexión automática con el espectrómetro a través del puerto serie.	Alta
RF-03	El sistema debe capturar los datos espectrométricos brutos procedentes de la muestra analizada.	Alta
RF-04	El sistema debe procesar los datos capturados utilizando un modelo de red neuronal preentrenado basado en TensorFlow.	Alta
RF-05	El sistema debe presentar los resultados derivados del análisis a través de la interfaz gráfica.	Alta
RF-06	El sistema debe habilitar la generación y exportación de un reporte detallado del análisis en formato PDF.	Alta
RF-07	El reporte generado debe contener obligatoriamente: fecha, resultados de la clasificación, métricas espectrométricas y métricas de confianza del modelo de IA.	Alta
RF-08	El sistema debe persistir de manera automática los datos de cada análisis ejecutado en una base de datos local.	Alta
RF-09	El sistema debe proporcionar una vista para la consulta del histórico de análisis previamente almacenados.	Media
RF-10	El sistema debe incorporar herramientas de filtrado sobre el histórico (por rango de fechas, tipo de resultado, etc.).	Media
RF-11	El sistema debe cargar el modelo de IA en memoria de forma automática durante la secuencia de inicio de la aplicación.	Alta

Tabla B.1: Tabla requisitos funcionales

Requisitos no funcionales

ID	Descripción	Prioridad
RNF-01	El tiempo máximo de ejecución de un análisis (desde la captura de datos hasta la renderización de resultados) no debe superar los 2 minutos.	Alta
RNF-02	<i>La aplicación debe estar desarrollada íntegramente en el lenguaje de programación Python.</i>	Alta
RNF-03	<i>El modelo de Inteligencia Artificial debe estar construido y ejecutado utilizando la biblioteca TensorFlow.</i>	Alta
RNF-04	La aplicación debe operar de forma estrictamente local en la estación de trabajo del usuario (sin dependencias de la nube para la inferencia).	Alta
RNF-05	El sistema debe garantizar la compatibilidad multiplataforma, soportando los sistemas operativos Windows, Linux y macOS.	Media
RNF-06	El sistema debe restringir la ejecución concurrente, permitiendo únicamente un análisis activo en un momento dado.	Alta
RNF-07	La interfaz gráfica de usuario debe basarse en principios de diseño minimalista, minimizando la carga cognitiva y facilitando la usabilidad.	Alta
RNF-08	El sistema operará bajo un modelo de acceso local directo, sin requerir módulos de autenticación de usuarios.	Baja
RNF-09	La persistencia de los datos históricos locales debe estructurarse de modo que facilite la extracción de datasets para el reentrenamiento futuro del modelo.	Media

Tabla B.2: Tabla requisitos no funcionales

B.4. Especificación de requisitos

A continuación, se presenta la especificación de los casos de uso principales del sistema HIVES, redactados conforme a los estándares habituales de la Ingeniería de Software.

Tabla B.3: CU-01 Analizar Muestra de Miel.

CU-01	Analizar Muestra de Miel
Versión	1.0
Autor	Diego Alonso Soria
Requisitos asociados	RF-01, RF-02, RF-03, RF-04, RF-05, RF-08, RNF-01, RNF-06
Descripción	El usuario inicia un análisis completo de la muestra de miel mediante el espectrómetro conectado al ordenador. El sistema captura los datos del espectrómetro, los procesa con el modelo de IA y presenta los resultados por pantalla
Precondición	■ El espectrómetro debe estar conectado físicamente mediante puerto serie ■ El modelo de IA debe encontrarse ubicado en el directorio models/ ■ El sistema debe encontrarse en estado inactivo (no puede haber otro análisis a la vez)
Acciones	1. El usuario ejecuta la aplicación de HIVES. 2. El usuario acciona el control “Iniciar Análisis”. 3. El sistema establece la conexión con el espectrómetro por el puerto serie seleccionado 4. El sistema inicia y completa la captura automatizada de los datos del espectrómetro. 5. El sistema computa las métricas de clasificación (categoría de la miel, índice de confianza del modelo, etc.) 6. El sistema renderiza los resultados en la interfaz gráfica. 7. El sistema persiste los datos del análisis en la base de datos local de forma transparente.

continúa en la página siguiente

continúa desde la página anterior

CU-01	Analizar Muestra de Miel
Postcondición	▪ Los datos brutos y procesados del análisis se quedan registrados en el histórico ▪ El usuario dispone de los resultados visuales en pantalla ▪ El sistema retorna a un estado de disponibilidad para nuevos análisis o exportaciones
Excepciones	▪ E1 Si no se detecta el espectrómetro, el sistema interrumpe el flujo, muestra un cuadro de diálogo de error y solicita verificar la conexión física. ▪ E2 Si se detecta un fallo en la carga del modelo de IA, el sistema notifica la incidencia y bloquea la ejecución de análisis. ▪ E3 Ante un error durante la adquisición de datos, el sistema aborta la operación y retorna al estado inicial de forma segura
Importancia	Alta

Tabla B.4: CU-02 Generar y Exportar Reporte PDF.

CU-02	Generar y Exportar Reporte PDF
Versión	1.0
Autor	Diego Alonso Soria
Requisitos asociados	RF-06, RF-07
Descripción	El usuario exporta los resultados de un análisis en formato PDF para su archivo documental o distribución externa. El reporte integra información técnica relevante recabada durante la prueba.
Precondición	▪ Debe existir, al menos, un análisis completo en memoria o registro en el histórico local. ▪ El sistema debe contar con permisos de escritura sobre el directorio de destino del sistema de archivos.

continúa en la página siguiente

continúa desde la página anterior

CU-02	Generar y Exportar Reporte PDF
Acciones	
	1. El usuario selecciona un análisis específico (ya sea el recién finalizado o uno extraído del histórico). 2. El usuario acciona el control “Generar Reporte PDF”. 3. El sistema extrae de la base de datos los campos requeridos: fecha y hora, resultado de clasificación, métricas del espectrómetro y métricas del modelo. 4. El sistema genera un documento PDF aplicando la plantilla estructurada predefinida. 5. El sistema invoca el cuadro de diálogo del sistema operativo para solicitar la ruta de guardado. 6. El sistema guarda el archivo PDF en el directorio especificado por el usuario.
Postcondición	
	■ El archivo PDF se genera exitosamente y se almacena en el disco del usuario. ■ El documento queda disponible para su consulta externa, impresión o distribución
Excepciones	
	■ E1 Si el sistema carece de permisos de escritura en la ruta especificada, se emite una alerta y se solicita seleccionar un directorio alternativo. ■ E2 En caso de fallo interno en la biblioteca de generación de PDFs, el sistema captura la excepción, notifica al usuario y registra el evento en el log de errores.
Importancia	Alta

Tabla B.5: CU-03 Consultar Histórico de Análisis.

CU-03	Consultar Histórico de Análisis
Versión	1.0
Autor	Diego Alonso Soria
Requisitos asociados	RF-09, RF-10

continúa en la página siguiente

continúa desde la página anterior

CU-03	Consultar Histórico de Análisis
Descripción	El usuario accede al repositorio de análisis previos registrados en el sistema, disponiendo de herramientas de filtrado para la localización de registros específicos.
Precondición	■ La base de datos local debe contener, como mínimo, un registro de análisis. ■ El usuario debe haber navegado hasta el módulo de histórico en la interfaz gráfica.
Acciones	1. El usuario acciona la pestaña o botón “Ver Histórico” en la navegación principal. 2. El sistema lanza una consulta a la base de datos para recuperar los registros almacenados. 3. El sistema despliega una tabla o listado resumen conteniendo: fecha, tipo de miel y nivel de confianza predictiva. 4. El usuario introduce parámetros en los controles de filtrado (rango temporal, categoría específica, umbral de confianza). 5. El sistema procesa los filtros y refresca la vista, mostrando exclusivamente los registros coincidentes. 6. El usuario selecciona una fila del histórico para desplegar su información extendida o derivar al flujo de exportación PDF (CU-02).
Postcondición	■ El usuario obtiene una vista refinada de los análisis históricos adaptada a sus criterios de búsqueda. ■ El sistema mantiene el estado de la vista para permitir la inspección en detalle de los registros listados.
Excepciones	■ E1 Si la base de datos se encuentra vacía, el sistema muestra un indicador visual (estado vacío) notificando la ausencia de datos históricos. ■ E2 Si la combinación de filtros aplicados resulta en un conjunto vacío, el sistema indica claramente que “No se han encontrado coincidencias”.
Importancia	Media

Resumen de Requisitos

El sistema HIVES ha sido formalmente especificado mediante 11 requisitos funcionales y 9 requisitos no funcionales, abarcando la totalidad de las características esenciales para garantizar el análisis automatizado de muestras de miel combinando espectrometría e Inteligencia Artificial

Los tres casos de uso principales (Analizar Muestra, Generar Reporte y Consultar Histórico) definen el núcleo operativo de la aplicación y se establecen como el pilar fundamental sobre el cual se desarrollará el diseño de la arquitectura software (Apéndice C) y se planificarán las pruebas funcionales (Apéndice D).

La correcta implementación de las especificaciones detalladas en este documento aseguran la consecución del objetivo principal del proyecto: reducir los costes y tiempos operativos del análisis apícola a través de la automatización inteligente, preservando en todo momento altos estándares de usabilidad, eficiencia algorítmica y fiabilidad.

Apéndice C

Especificación de diseño

C.1. Introducción

Una vez realizada la exhaustiva formalización de los requisitos funcionales y no funcionales de la aplicación, así como la definición de los casos de uso que acotan las interacciones del actor principal, el presente apéndice se centra en las especificaciones de diseño interno. Estas especificaciones recogen y detallan las directrices técnicas, la arquitectura de software, los paradigmas de concurrencia y la pila tecnológica (*tech stack*) que sustentan la materialización del proyecto. El objetivo de este capítulo es proporcionar una justificación instrumental y algorítmica profunda para cada decisión adoptada, demostrando la viabilidad técnica y el rigor aplicados en la ingeniería del sistema HIVES.

C.2. Diseño de datos y capa de persistencia

En lo referente a la gestión de la información del proyecto, la arquitectura de datos se ha concebido bajo el principio de separación de responsabilidades y la inmutabilidad de la lectura física. La información se divide en dos dominios lógicos principales: los datos empíricos obtenidos en bruto que han de ser almacenados para garantizar la trazabilidad pericial, y los datos procesados y normalizados que son consumidos directamente por los tensores del modelo predictivo.

Para materializar la capa de persistencia local se ha implementado el motor de bases de datos relacional embebido **SQLite**. La adopción de esta tecnología, frente a alternativas cliente-servidor tradicionales como Post-

greSQL o MySQL, se fundamenta en un análisis estricto de los requisitos de despliegue. Al ser HIVES una aplicación de escritorio de ámbito monopuesto, SQLite [26] permite encapsular el esquema relacional completo, los índices y los registros en un único archivo físico dentro del sistema de ficheros del equipo anfitrión. Esta característica garantiza una portabilidad absoluta y elimina la fricción operativa que supondría obligar al usuario final a aprovisionar, configurar y mantener un demonio de base de datos externo. Adicionalmente, SQLite asegura el cumplimiento de las propiedades transaccionales ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad)[25], protegiendo el histórico de análisis frente a corrupciones de escritura derivadas de cortes de suministro eléctrico o caídas abruptas del sistema.

Desde el punto de vista del rendimiento computacional, se ha prescindido deliberadamente de mapeadores objeto-relacional (ORM) de alto nivel (como SQLAlchemy). La interacción con la base de datos se realiza mediante la inyección directa de consultas SQL puras parametrizadas. Esta decisión arquitectónica minimiza el *overhead* de traducción de objetos, reduce el consumo de memoria RAM de la aplicación y otorga un control determinista sobre los tiempos de inserción, lectura y filtrado del histórico, un factor de suma relevancia para garantizar la inmediatez de respuesta exigida por la interfaz gráfica.

A nivel de esquema, una de las decisiones de diseño más críticas reside en el tratamiento de los 18 canales espectrales capturados por el sensor AS7265x. En lugar de generar un esquema altamente acoplado con 18 columnas numéricas individuales, los valores se agrupan y se almacenan en formato de *array* serializado (típicamente mediante notación JSON) dentro de una única columna denominada **espectro_raw** en la tabla de **muestras**. Este patrón de diseño estructural, conocido como *Serialized LOB (Large Object)* [6], proporciona una notable flexibilidad frente al cambio. Si en futuras iteraciones del proyecto se decide actualizar el hardware espectrométrico a un sensor de 36 o 72 canales, el esquema relacional de la base de datos permanecerá inalterado, requiriendo modificaciones únicamente en la capa de serialización lógica. Además, la preservación íntegra del espectro en bruto actúa como un mecanismo de auditoría inmutable: ante cualquier actualización futura de los algoritmos de normalización, el conjunto de datos empírico original podrá ser extraído intacto para proceder a nuevos ciclos de reentrenamiento.

A continuación, se presenta el modelo conceptual de la base de datos:

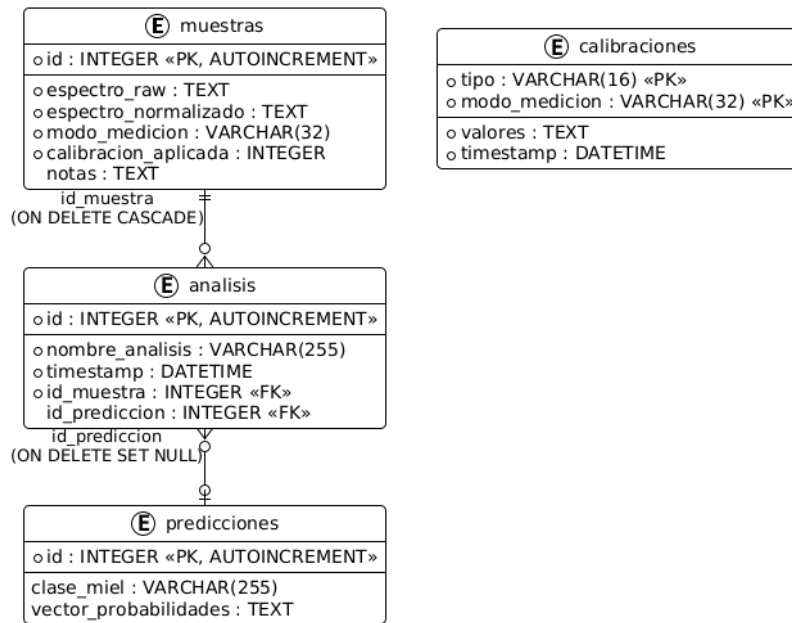


Figura C.1: Modelo conceptual y diagrama de tablas de la base de datos local. Fuente: Elaboración Propia.

En contraposición a los datos en bruto, los datos inyectados en el modelo de Inteligencia Artificial requieren un preprocesamiento algorítmico estricto. Las lecturas provenientes de `espectro_raw` son extraídas y sometidas a un *pipeline* de normalización matemática para estabilizar las varianzas entre canales, mitigar el ruido analógico de los fotodiodos y acotar los gradientes numéricos, garantizando así la convergencia de la red neuronal.

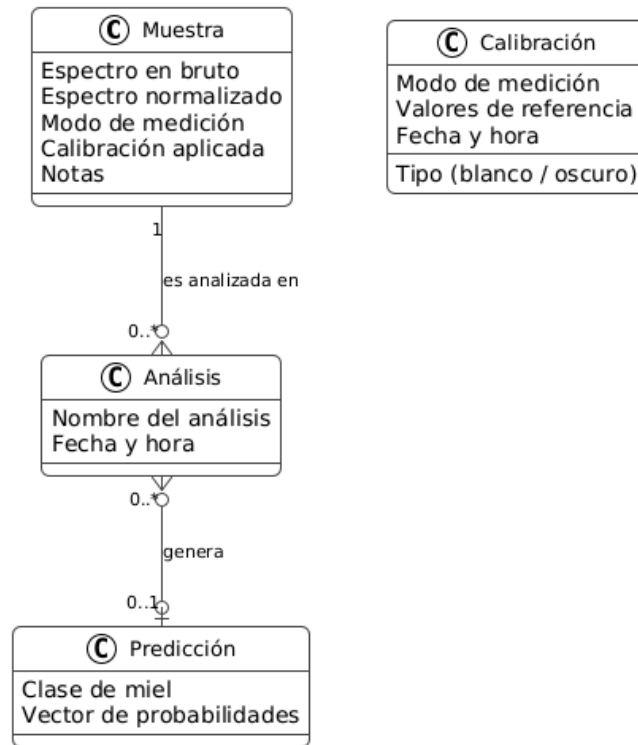


Figura C.2: Diagrama Entidad-Relación de la base de datos local. Fuente: Elaboración Propia.

C.3. Diseño procedimental y flujo de ejecución

Descripción analítica del pipeline de procesamiento

El diseño procedimental del sistema HIVES define el ciclo de vida del dato, desde su conversión analógico-digital en la capa física hasta su representación categórica en la interfaz. El flujo principal de ejecución sigue una arquitectura de procesamiento en tubería (*pipeline*) conformada por seis etapas lógicas secuenciales: (1) captura sensorica asíncrona, (2) almacenamiento estructural en bruto, (3) normalización vectorial, (4) inferencia probabilística mediante Inteligencia Artificial, (5) generación programática de reportes y (6) persistencia relacional del dictamen final.

El proceso global se desencadena cuando el actor principal solicita el análisis de una muestra de miel a través de la interfaz gráfica. La petición viaja hacia la pasarela hardware (el microcontrolador Arduino Uno), que

efectúa la adquisición del espectro óptico temporalmente promediado. Una vez recibida la trama de datos, la capa de software asume el control absoluto del flujo, garantizando que cada etapa se complete de forma atómica antes de derivar el flujo de control hacia la siguiente subrutina.

Módulo de captura de espectro y telemetría

Este módulo es el responsable de gestionar la capa de enlace y comunicación de bajo nivel con el hardware espectrométrico. A nivel electrónico, el sensor **SparkFun Triad AS7265x** es gobernado por la lógica interna del microcontrolador (programado en lenguaje C/C++) a través de un bus síncrono **I²C**, gestionado externamente mediante la librería **SparkFun_AS7265X_Arduino_Library**. Sin embargo, la comunicación entre este hardware y el ordenador anfitrión exige un diseño procedimental más sofisticado en la capa de software:

- **Gestión de puerto serie virtual:** El establecimiento de la comunicación bidireccional se orquesta empleando la biblioteca **PySerial**[13]. Su utilización permite instanciar un canal de comunicación asíncrono sobre el estándar USB, proveyendo una API abstracta que oculta las complejidades de implementación subyacentes entre el gestor de dispositivos de Windows, las interfaces **tty** de Linux y los entornos macOS.
- **Gestión de búferes y sincronización:** Previo a la solicitud de cualquier lectura, el controlador vacía activamente los búferes de entrada y salida del puerto serie (*flush*). Esta acción procedimental previene la lectura de tramas huérfanas o espectros residuales (*stale data*) pertenecientes a ejecuciones abortadas con anterioridad, garantizando la pureza temporal de la nueva muestra.
- **Validación estructural por dimensionalidad:** Tras la adquisición de la cadena de texto serie, los datos son decodificados (típicamente desde UTF-8) y troceados mediante delimitadores. El módulo realiza una comprobación estricta de cardinalidad: la matriz resultante debe contener exactamente 18 campos numéricos de coma flotante representativos de las bandas ópticas; cualquier desviación (producto de una caída de tensión o ruido eléctrico en el bus) provoca el descarte inmediato de la trama y la invocación de una rutina de reintento.

Módulo de almacenamiento de entrada (*Data Ingestion*)

Validada la captura espectral, la responsabilidad se delega al módulo de ingesta, encargado de la primera consolidación permanente de la información:

- **Transacción atómica:** El guardado del array en la tabla `muestras` (columna `espectro_raw`) se ejecuta empaquetado en una transacción atómica explícita de SQLite (`BEGIN TRANSACTION`). Esto asegura que, si el sistema colapsa durante la escritura del estado de calibración u otros metadatos asociados, la inserción se deshará (*rollback*) por completo, impidiendo la generación de registros espectrales huérfanos.
- **Generación de *Primary Keys*:** El motor de persistencia asigna un identificador único correlativo (clave primaria autoincremental) que acompañará al vector de datos durante el resto de su ciclo de vida en memoria, sirviendo de ancla referencial para el resultado analítico posterior.

Módulo de normalización matemática y acondicionamiento tensorial

Con el fin de compatibilizar la naturaleza discreta de los datos capturados con los requisitos formales de las arquitecturas de redes neuronales, este componente ejecuta transformaciones algebraicas críticas:

- **Escalado numérico:** Los valores de intensidad de los fotodiodos presentan magnitudes brutas dispares dependientes del tiempo de integración. Se aplica un algoritmo de escalado (Min-Max o estandarización *Z-score*[\[10\]](#), según los hiperparámetros serializados del modelo) para reducir las magnitudes al dominio de convergencia óptima de las funciones de activación, evitando la explosión o desvanecimiento del gradiente[\[3\]](#).
- **Reestructuración matricial (*Reshape*):** El vector unidimensional de 1×18 elementos es redimensionado dinámicamente para transformarse en un tensor de lote unitario (estructura `[1, 18]`), arquitectura obligatoria requerida por la capa `InputLayer` del grafo de TensorFlow.
- **Transición estricta de tipos de datos (*Casting*):** La lista genérica generada por el intérprete de Python es convertida forzosamente a

una estructura matricial empleando el tipo de dato primitivo coma flotante de precisión simple (`float32`). La adopción de `float32` frente a `float64` reduce el tamaño del tensor a la mitad, aliviando la carga en los cachés del procesador y manteniendo un alineamiento directo con las implementaciones nativas optimizadas de los binarios compilados de TensorFlow bajo lenguaje C++.

Módulo de clasificación probabilística (Núcleo de Inteligencia Artificial)

Este componente encapsula la lógica matemática y algorítmica central del producto. La complejidad de este módulo se fundamenta en un extenso proceso previo de I+D en ingeniería de datos, materializado originalmente en **Jupyter Notebooks** ejecutados sobre aceleradores de hardware (GPU Nvidia) mediante la plataforma en la nube **Google Colab**. Dicho entorno garantizó la potencia de cálculo masiva necesaria para emplear bibliotecas aceleradas como **CuML** de Nvidia.

El desafío primario abordado consistió en la superación de la escasez crítica del dataset empírico, limitado a 42 muestras de campo. La aplicación directa de modelos profundos sobre conjuntos de este tamaño conduce a escenarios de sobreajuste (*overfitting*) extremos. Para mitigarlo, se programó un subsistema de aumento de datos basado en **difusión gaussiana**. Inyectando ruido normal multivariante controlado sobre las firmas reales, se sintetizó el `dataset_completo_40036_v2.csv`, un conjunto de cerca de 40 000 muestras que retiene la topología y covarianza de las clases botánicas originales, proporcionando la masa crítica computacional requerida para el descenso de gradiente.

A nivel procedimental de despliegue, el modelo integrado en la aplicación de escritorio final opera basándose en una topología optimizada de **destilación de conocimiento** (*knowledge distillation*) [11], ejecutada en dos estadios formativos:

1. **Capa de maestría algorítmica:** Un modelo de ensamblado de aceleración de gradiente basado en árboles de decisión (**XGBoost**[4]), configurado y optimizado en profundidad mediante validación cruzada estratificada (*GridSearchCV* de 5 pliegues), procesó el dataset para generar distribuciones probabilísticas en lugar de predicciones de clase rígidas. Estos vectores enriquecidos, denominados *soft labels*, contienen la topología difusa del problema espectral (por ejemplo, identificando similitudes sutiles entre una Miel de Bosque y un Milflores).

2. **Capa de imitación tensorial:** Posteriormente, una **red neuronal densa** implementada bajo el *framework* **TensorFlow/Keras** —con una arquitectura de descompresión y compresión dimensional progresiva de $128 \rightarrow 64 \rightarrow 32 \rightarrow 5$ neuronas y funciones de activación ReLU, ReLU, ReLU y Softmax respectivamente — fue entrenada exclusivamente para imitar las predicciones suavizadas del XGBoost. Esta técnica permite transferir la robustez estructural y la resistencia al ruido propia de los bosques aleatorios hacia una estructura de grafos neuronales deterministas.

El archivo binario resultante de este proceso, `mejor_modelo.keras`, es cargado en memoria RAM en tiempo de arranque de la aplicación. Para asegurar una decodificación agnóstica a la versión del modelo, el sistema se apoya en un mapeador dinámico inyectado en un archivo `clases.json`, que traduce de forma determinista el nodo de la capa `softmax` con máxima excitación algorítmica (*argmax*[9]) a la cadena de texto nominativa de la clase botánica real.

Módulo de emisión documental y reportes

Garantizando la dimensión pericial y de trazabilidad funcional del sistema en entornos agroalimentarios comerciales, el diseño procedimental contempla un subcomponente especializado en la emisión estandarizada de informes de resultados. La maquetación programática de la memoria técnica en formato PDF es ejecutada por la biblioteca **FPDF**.

La justificación de esta herramienta arquitectónica resulta clave: mientras que otras alternativas del ecosistema Python (como WeasyPrint o pdfkit) requieren la renderización previa de un DOM en formato HTML/CSS invocando motores pesados o ejecutables del sistema operativo, FPDF interactúa directamente con las primitivas de dibujo y flujos del estándar PDF. Esta decisión reduce la huella de memoria (*memory footprint*) del proceso de exportación a márgenes casi nulos y evita dependencias de bibliotecas dinámicas (DLLs/.so) de terceros, lo que facilita enormemente el empaquetado final del ejecutable de la aplicación.

Módulo de persistencia analítica de resultados

La fase terminal del ciclo de procesamiento consiste en la consolidación relacional del dictamen algorítmico, operando nuevamente de forma atómica sobre la base de datos local:

- **Inserción de veredictos:** Se registra una nueva tupla en la tabla **predicciones**, almacenando tanto la clase categórica ganadora como el vector probabilístico completo (el nivel de confianza o certidumbre arrojado por la capa **softmax**).
- **Cierre referencial:** Se inserta la operación en la tabla de **analisis**, vinculando estáticamente a través de claves foráneas (*Foreign Keys*) la predicción obtenida con el espectro original almacenado en la etapa inicial. A este registro maestro se le adjunta el sello temporal (*timestamp*), cerrando la cadena de custodia digital del análisis espectrométrico.

Resiliencia del flujo y gestión estructurada de excepciones

La ingeniería de un sistema de criticidad analítica exige una tolerancia a fallos (*fault tolerance*) diseñada explícitamente y no delegada al manejo estándar de excepciones del intérprete. El sistema implementa políticas de *fail gracefully* (falla elegante), asegurando que ninguna perturbación de hardware o software provoque el colapso catastrófico o cierre no autorizado de la aplicación principal.

Tabla C.1: Mapeo de criticidad de escenarios de error y estrategias de mitigación en el sistema HIVES.

Fallo o Excepción	Capa de Origen	Tratamiento Lógico y Procedimental
<i>Timeout Serie o Pérdida de HW</i>	Módulo de enlace físico (Captura)	Aborto del hilo de lectura, purgado de <i>buffers</i> , emisión de señal asíncrona de error al <i>GUI</i> para notificación contextual al operador, registro exhaustivo en <i>logger</i> del sistema.
Bloqueo SQL (<i>Database Locked</i>)	Motor Relacional (Persistencia)	Backoff algorítmico con límite de reintentos. Si persiste el fallo de acceso I/O, el estado de la muestra se revierte en memoria (rollback) notificando incapacidad de escritura.
Dimensión tensorial malformada	Preprocesamiento (Normalización)	Bloqueo preventivo de la inferencia. Intercepción de excepción matricial y descarte de la firma espectral antes de ser inyectada en el modelo de Inteligencia Artificial.
Ausencia de grafo de IA (.keras)	Motor Predictivo (Clasificación)	Captura en inicialización <i>early-fail</i> . Inhabilitación visual del botón de escaneo en la interfaz, solicitando reintegro del binario modelo.

Diagramas formales de flujos lógicos y estructurales

Con el fin de proveer una modelización visual inequívoca del ciclo de vida detallado, se incluyen los esquemas formales generados en notación de Lenguaje Unificado de Modelado (UML):

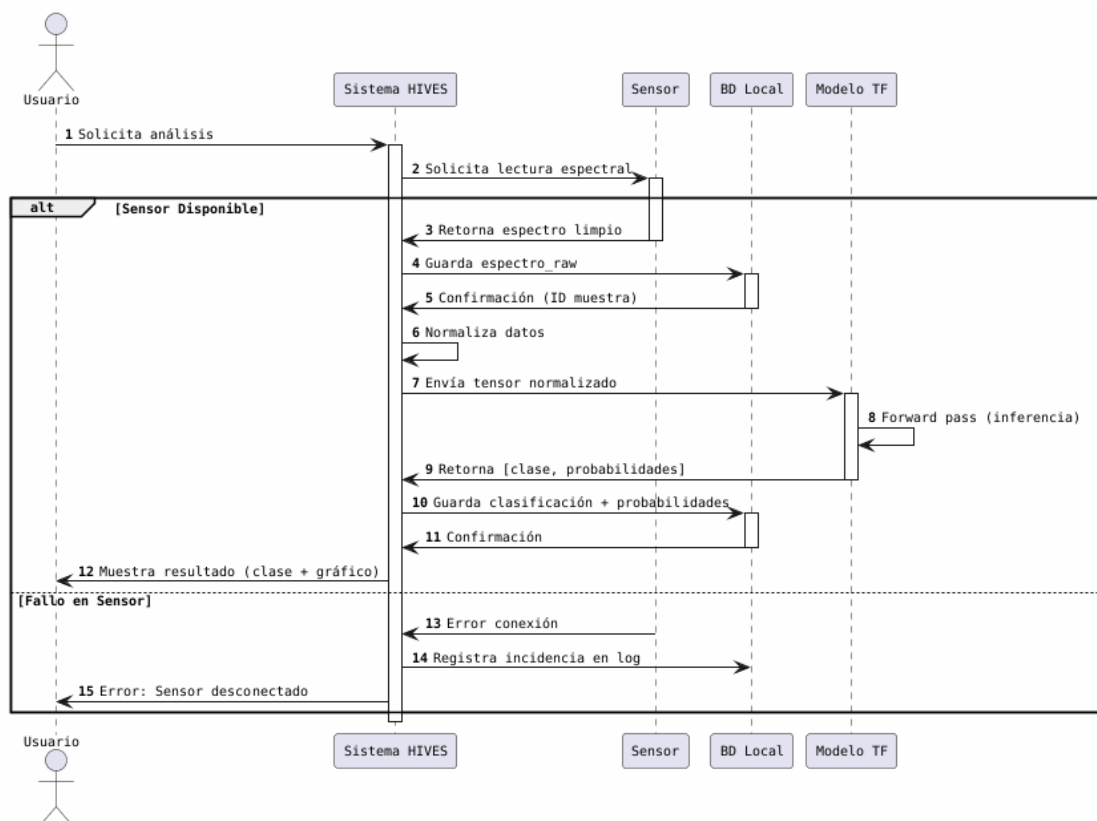


Figura C.3: Diagrama de secuencia UML del análisis de una muestra de miel. Se detalla la sincronía de las llamadas, los tiempos de vida de los objetos y la delegación a hilos de trabajo paralelos. Fuente: Elaboración Propia.

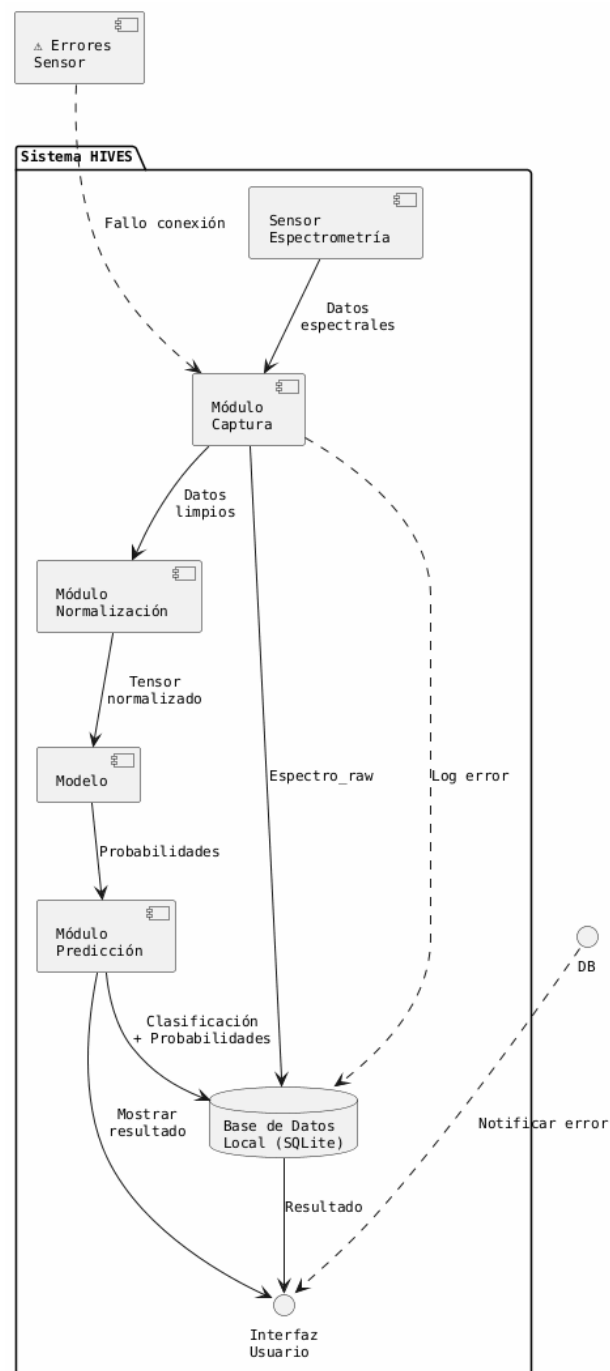


Figura C.4: Diagrama de flujo de datos (DFD). Identificación de procesos, almacenes de persistencia temporales e interfaces de intercambio de estructuras de datos a través del sistema HIVES. Fuente: Elaboración Propia.

C.4. Diseño arquitectónico y aislamiento de capas

A nivel estructural macrosistemático, el código base ha sido modelado y codificado implementando estrictamente el patrón arquitectónico **Modelo-Vista-Controlador (MVC)** [8]. Esta metodología de ingeniería de *software* rige la distribución del código fuente, garantizando el principio de Responsabilidad Única (*Single Responsibility Principle*) y un bajo acoplamiento estructural.

La adopción de **Python 3.10** como único lenguaje integrador, operando como núcleo tanto para el entorno estadístico del modelo como para el *backend* aplicativo, justifica esta aproximación MVC, posibilitando la intercomunicación fluida en espacio de memoria compartida sin tener que lidiar con complejos *bindings* o puentes de lenguajes externos (como ocurriría si el interfaz se desarrollase en C# WPF o JavaFX frente a un motor predictivo en Python).

Las tres responsabilidades modulares operan bajo la siguiente distribución de diseño:

- **Capa de Presentación (Vista):** Materializada a través de las primitivas de **PyQt6**. Es una capa intencionadamente "tonta", desprovista de cualquier operación aritmética o lógica de negocio. Se limita a instanciar los *widgets* (botones, gráficos, tablas), capturar las interrupciones del operador y redibujar componentes al recibir dictámenes calculados. También se planteó como alternativa el uso de PySide6 como solución a la licencia de PyQt6, pero se descartó a favor de PyQt6, para el que se encontró más documentación técnica que facilitó su implementación.
- **Capa de Orquestación (Controlador):** Actúa como el centro neurálgico (*Brain*) de la aplicación. Inicializa las conexiones periféricas de puerto serie (**PySerial**), valida el estado de la comunicación y coordina el tráfico de información entre lo que el usuario solicita y los servicios que deben ejecutarse.
- **Capa de Entidad lógicas (Modelo):** Sub-arquitectura que envuelve por completo toda la semántica del acceso a los ficheros locales, el control transaccional del **SQLite** y la encapsulación compleja del motor matemático de **TensorFlow**. Cualquier cambio futuro en el algoritmo

de IA requerirá alteraciones únicas en el Modelo, permaneciendo totalmente opaco para las otras dos capas.

Patrón reactivo de comunicación: *Signals and Slots*

Para materializar físicamente la separación MVC dentro del ecosistema Qt, el sistema implementa un enrutamiento de eventos mediante el potente paradigma de *Signals and Slots* [21] (señales y ranuras). Este mecanismo constituye una implementación segura para tipos (*type-safe*) del clásico patrón de diseño *Observer* (Observador).

A diferencia de las arquitecturas basadas en *callbacks* encadenados — que frecuentemente derivan en problemas de mantenimiento y referencias circulares de memoria—, en PyQt6 un emisor que dispara una señal (*signal*) ignora completamente quién está escuchando o si siquiera existe algún receptor suscrito. Simétricamente, la ranura receptora (*slot*) ejecuta su bloque de código sin necesidad de mantener un puntero al objeto originario. En HIVES, el Controlador suscribe la funcionalidad analítica a la señal de *click* de la Vista, mientras que la Vista vincula sus paneles de actualización gráfica a las señales de finalización arrojadas por el Controlador. Este aislamiento bidireccional garantiza un código limpio, auditable y de extrema facilidad para la inserción de pruebas unitarias.

Gestión avanzada de concurrencia y subprocesos (*QThread*)

La ejecución de cálculos algebraicos complejos (como el avance secuencial a través de múltiples capas tensoriales en TensorFlow) y los bloqueos síncronos por entrada/salida de hardware (I/O, lectura asíncrona del puerto serie) requieren tiempos de computación de varios milisegundos o incluso segundos. Si estas lógicas se ejecutasen en el hilo predeterminado de la aplicación de Python, el bucle principal de eventos gráficos (*Main Event Loop*) quedaría interrumpido indefinidamente, provocando la indeseada congelación visual de la interfaz (*GUI freezing*) y la potencial marcación del programa como "No responde" por parte del sistema operativo anfitrión.

Para erradicar esta problemática estructural, se implementa una programación concurrente rígida delegando las funciones pesadas hacia hilos de trabajo paralelos (*Worker Threads*) administrados por la clase de interfaz **QThread**[18]. Se ha preferido el uso de **QThread** por encima de las librerías genéricas nativas de Python (**threading** o **asyncio**) por su seguridad intrínseca. En PyQt, el mecanismo de *Signals and Slots* está

diseñado para ser seguro entre hilos (*thread-safe*): cuando un hilo secundario de procesamiento algorítmico termina su tarea, emite una señal que es encolada automáticamente y de manera segura en el bucle del hilo principal (*Qt::QueuedConnection*), protegiendo el espacio de memoria y garantizando que las modificaciones gráficas (únicamente permitidas en el hilo principal por el *framework* Qt) ocurran sin colisiones ni condiciones de carrera (*race conditions*).

C.5. Herramientas de gestión del ciclo de vida, calidad de código y empaquetado

La garantía del éxito de un proyecto de ingeniería de software moderno no depende exclusivamente de las elecciones arquitectónicas internas, sino de la robustez de las herramientas (*tooling*) que acompañan el ciclo de vida del producto, desde su concepción en los requerimientos hasta su distribución como binario compilado.

- **Estrategia metodológica y adaptación ágil:** La ejecución temporal y la monitorización se ha llevado a cabo bajo el marco prescriptivo **Scrum**. Dado el paradigma unipersonal de este TFG, los roles han sido readaptados sin perder el rigor temporal (*timeboxing*) de *sprints* semanales. Toda la traducción algorítmica de requisitos y el control analítico del trabajo pendiente (*Product Backlog*) se ha administrado en la plataforma en la nube **Jira**. Mediante la estimación de complejidad a través de *Story Points* y la evaluación de gráficas *burn-down*, se garantizó el mantenimiento predecible de la velocidad de desarrollo.
- **Entorno integral y versionado de ramas:** Como entorno de desarrollo integrado principal se seleccionó **Visual Studio Code**, provisto de analizadores estáticos (*linters*) específicos para el cumplimiento de guías de estilo Python (PEP-8). El salvaguardado progresivo de las versiones estables de software y los experimentos algorítmicos paralelos se gestionó ininterrumpidamente mediante **Git**, consolidando el repositorio seguro y las revisiones en **GitHub**.
- **Análisis paramétrico de calidad:** Para prevenir la inclusión sistémica de deuda técnica (*Technical Debt*), evitar la complejidad ciclomática excesiva y asegurar que los componentes estuvieran libres de vulnerabilidades estructurales comunes (*security hotspots*), la base completa

de código fuente fue sometida a auditorías automatizadas periódicas orquestadas mediante la plataforma **SonarQube**.

- **Generación documental algorítmica:** Asegurando que la documentación técnica visual nunca quede desfasada con respecto a las modificaciones del código fuente, toda la esquemática UML desplegada en el presente apéndice ha sido declarada como código en texto plano utilizando la herramienta **PlantUML**, lo que permite su versionado natural junto a los módulos del *software*. La redacción tipográfica integral de la presente memoria técnica se consolidó en el compilador estandarizado **LaTeX** sobre la plataforma **Overleaf**.
- **Despliegue operativo y empaquetado de distribución:** La barrera final de usabilidad del software desarrollado exige que el usuario industrial no requiera de complejos manuales de configuración de variables de entorno ni de instalaciones de intérpretes subyacentes. Este reto de *deployment* se abordó íntegramente configurando reglas precisas en **PyInstaller**[16]. Esta herramienta realiza un análisis profundo de dependencias del árbol de importaciones (AST) y compila un paquete binario (*executable*). PyInstaller embebe dinámicamente un microintérprete optimizado de Python, los grafos serializados de TensorFlow (cuyo enrutado relativo de directorios exige configuraciones personalizadas del espacio temporal **_MEIPASS**), el binario precompilado de la base de datos SQLite y las rutinas gráficas de PyQt6 en un único archivo ejecutable autónomo. Este diseño procedimental culmina la visión técnica del sistema HIVES proporcionando una solución tecnológica directa, contenida, de máxima eficacia operativa y lista para entornos de producción agroalimentarios.

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

El presente apéndice conforma el Manual del Programador y de Pruebas del sistema HIVES. Su objetivo principal es documentar la arquitectura a nivel de código, la organización del repositorio, el entorno necesario y los procedimientos de validación del software seguidos. Esta documentación está orientada a facilitar la mantenibilidad, futura escalabilidad y el despliegue del sistema por parte de otros ingenieros o desarrolladores que deseen auditar o continuar el proyecto.

D.2. Estructura de directorios

El repositorio del proyecto ha sido organizado siguiendo el patrón estándar para aplicaciones modulares en Python, separando claramente la lógica de negocio, los datos, los modelos de Inteligencia Artificial y los recursos estáticos. La jerarquía principal de directorios es la siguiente:

```
Trabajo-Fin-de-Grado-2025-2026/
├── arduino
│   └── LecturasParaInterfaz.ino ..... Firmware del sensor (C++)
├── release ..... Contiene los ejecutables de Windows y Linux.
├── data ..... Almacena datos y logs
│   ├── data.db ..... Base de datos SQLite (persistencia local)
│   └── hives.log ..... Logs para diagnóstico de errores
```

models	Lógica de entrenamiento y modelos
mejor_modelo.keras	Modelo en producción (TensorFlow)
clases.json	Nombres de clase en inferencia
mejor_modelo_final.keras	Modelo anterior (descartado)
pruebaColab.ipynb	Notebook activo de entrenamiento
pruebaTensorDataSetCorregido.ipynb	Notebook anterior (descartado)
src/	
main.py	ENTRY POINT de la aplicación
constants.py	
sensor.py	
runtime_hook.py	
db/	Inicializador de la base de datos
__init__.py	
seeder.py	
hooks/	Empaquetado de ML con PyInstaller
hook-keras.py	
hook-matplotlib.py	
hook-tensorflow.py	
hives/	CÓDIGO ACTIVO (paquete hives.*)
__init__.py	
constants.py	Constantes globales del proyecto
core/	
__init__.py	
sensor.py	Comunicación serie (SerialReader)
database.py	Inicialización del esquema de la BD
paths.py	Variables de rutas
gui/	Capa de presentación (Vista MVC)
__init__.py	
main_window.py	Ventana principal (SpectroControlUI)
widgets.py	Canvas espectral (SpectralCanvas)
workers.py	Hilos QThread asíncronos
dialogs.py	Diálogos modales
inference/	
__init__.py	
model.py	Carga y ejecución del modelo
reports/	
__init__.py	
pdf_report.py	Generación de informes PDF
tests/	Pruebas automatizadas (pytest)

— conftest.py.....	<i>Fixtures compartidos (BD, mocks, ventana)</i>
— test_database.py	
— test_dialogs.py	
— test_inference.py	
— test_main_window.py	
— test_pdf_report.py	
— test_sensor.py	
— test_widgets.py	
— test_workers.py	
— build_linux.sh.....	<i>Construcción en Linux</i>
— build_macos.sh.....	<i>Construcción en macOS</i>
— build_windows.ps1.....	<i>Construcción en Windows</i>
— coverage.xml.....	<i>Reporte de cobertura (pytest)</i>
— pyproject.toml.....	<i>Configuración de pytest</i>
— requirements.txt.....	<i>Dependencias Python</i>
— sonar_project.properties.....	<i>Configuración SonarQube</i>
— HIVES.spec.....	<i>Configuración para PyInstaller</i>
— README.md.....	<i>Documentación del repositorio</i>

D.3. Manual del programador

Este apartado detalla las dependencias y las decisiones de implementación adoptadas durante el desarrollo del sistema.

Entorno de desarrollo y dependencias

El núcleo de la aplicación ha sido desarrollado íntegramente en **Python 3.10**, utilizando **Visual Studio Code** como Entorno de Desarrollo Integrado (IDE) y **Git** para el control de versiones, alojando el repositorio en GitHub.

El proyecto hace uso de las siguientes librerías de terceros:

- **PyQt6:** *framework* para la construcción de la interfaz gráfica de usuario (GUI) y la gestión de hilos mediante **QThread**.
- **TensorFlow:** biblioteca para la carga del modelo de red neuronal y la ejecución de la inferencia en producción.
- **Scikit-learn:** biblioteca empleada durante la fase de experimentación para el entrenamiento y evaluación de clasificadores alternativos, incluyendo **Random Forest**.

- **cuML**: biblioteca utilizada durante la fase de experimentación para el entrenamiento y comparativa de clasificadores de alto rendimiento, incluyendo nuevamente **Random Forest**.
- **XGBoost**: biblioteca empleada durante la fase de experimentación para el entrenamiento y evaluación de clasificadores alternativos, incluyendo **XGBClassifier**, que terminó integrándose como la primera etapa de procesamiento dentro de la arquitectura híbrida definitiva.
- **PySerial**: módulo empleado para establecer la comunicación de bajo nivel con el Arduino Uno a través del puerto serie.
- **FPDF2**: librería para la maquetación y generación de los reportes en formato PDF.

Adicionalmente, se hace un uso extensivo de la librería estándar de Python (`sqlite3`, `os`, `sys`, `time`, `datetime`).

Detalles de implementación: concurrencia y base de datos

Para evitar la congelación de la interfaz gráfica (*GUI freezing*) durante la lectura del puerto serie y la inferencia del modelo, se ha implementado un sistema de concurrencia basado en **QThread** de PyQt6. Este hilo secundario se levanta exclusivamente al iniciar el análisis y emite una señal a la vista principal una vez obtenidos los resultados, manteniendo la interfaz reactiva en todo momento.

En cuanto a la persistencia de datos, se ha prescindido de mapeadores objeto-relacional (ORM) en favor de consultas SQL puras sobre SQLite. Esta decisión minimiza el consumo de recursos y otorga un control optimizado sobre las sentencias de inserción y filtrado del histórico. El esquema de la base de datos, compuesto por las tablas **muestras**, **predicciones**, **analisis** y **calibraciones**, ha sido inicializado mediante un script de *inicialización* y *seeding* que garantiza la creación de la estructura relacional en el primer arranque de la aplicación.

D.4. Pruebas del sistema

Para asegurar la calidad, fiabilidad y robustez de la aplicación, se ha diseñado una batería de pruebas centrada en los componentes críticos del sistema.

Batería de pruebas automatizadas

El proyecto cuenta con una suite de pruebas unitarias y de integración implementada mediante el *framework* **pytest**[17], ejecutable de forma completa con el siguiente comando desde la raíz del repositorio:

```
pytest tests/
```

La suite se organiza en ocho módulos de test independientes, cada uno orientado a un componente específico de la arquitectura:

- **test_database.py**: verifica la correcta creación del esquema de la base de datos y la integridad referencial entre las tablas **muestras**, **predicciones**, **analisis** y **calibraciones**.
- **test_sensor.py**: prueba el módulo de comunicación serie, comprobando el comportamiento ante desconexiones abruptas, *timeouts* y tramas de datos incompletas o corruptas.
- **test_inference.py**: valida que el módulo de clasificación recibe tensores con la dimensionalidad y normalización correctas y emite predicciones coherentes con la arquitectura del modelo.
- **test_main_window.py**: prueba la lógica de la ventana principal, incluyendo el ciclo de análisis completo y la gestión de estados de la interfaz.
- **test_widgets.py**: comprueba el funcionamiento del canvas espectral (**SpectralCanvas**) y su renderizado.
- **test_workers.py**: verifica el comportamiento de los hilos **QThread**, asegurando que las señales se emiten correctamente al hilo principal una vez completada la operación.
- **test_dialogs.py**: valida los diálogos modales de la aplicación y su interacción con el flujo principal.

- `test_pdf_report.py`: comprueba la generación de informes PDF, verificando que el documento resultante contiene los campos requeridos por el requisito RF-07.

Estrategia de *mocking* y *fixtures*

Dado que el sistema interactúa con hardware externo y con dependencias de gran peso computacional, la suite de pruebas hace un uso extensivo de *mocking*^[7] para aislar cada componente y garantizar que los tests son reproducibles en cualquier entorno, incluyendo entornos de integración continua sin hardware físico disponible. Todas las *fixtures* compartidas se definen de forma centralizada en `tests/conftest.py`:

- `mem_db`: conexión SQLite en memoria (`:memory:`) con el esquema completo inicializado. Empleada en los tests de base de datos puros, garantiza el aislamiento total entre ejecuciones.
- `test_db`: ruta a un fichero `.db` temporal con todas las tablas creadas, útil para los tests que requieren una base de datos persistente durante el ciclo de vida del test.
- `mock_serial`: parchea `serial.Serial` globalmente, simulando una conexión abierta con comportamientos configurables por cada test. Permite reproducir escenarios de desconexión, *timeout* o recepción de tramas corruptas sin necesidad del hardware real.
- `mock_reader`: instancia un `SerialReader` del paquete activo con la conexión serie ya vinculada al `mock_serial`, eliminando la necesidad de reconfigurar el *mock* en cada test individual.
- `mock_model`: simula un modelo TensorFlow mediante `MagicMock`, devolviendo un vector de 5 probabilidades uniformes y la lista de las 5 clases reales en nuestro caso. Permite ejecutar el flujo completo de inferencia sin cargar TensorFlow en memoria.
- `window`: instancia completa de `SpectroControlUI` con la ruta de base de datos, el modelo y los puertos serie parcheados, registrada con `qtbott.addWidget()` para su limpieza automática al finalizar cada test.

Las dependencias que no se mockean son aquellas que ya incorporan mecanismos de resiliencia propios: `inference/model.py` gestiona internamente

el `ImportError` de TensorFlow, y tanto `widgets.py` como `pdf_report.py` disponen de rutas de *fallback* ante la ausencia de **matplotlib**, por lo que su comportamiento queda cubierto de forma indirecta por los tests funcionales.

Pruebas de integración y hardware

Dada la naturaleza del proyecto, las pruebas de integración se centran en verificar la correcta comunicación entre el software y el hardware externo. Se han realizado pruebas de estrés y validación sobre el puerto serie (mediante **PySerial**) para garantizar que el sistema maneja correctamente las desconexiones abruptas del Arduino Uno, los tiempos de espera agotados (*timeouts*) y la recepción de tramas de datos incompletas o corruptas. Asimismo, se ha verificado la correcta interacción entre la lógica de la aplicación y la base de datos SQLite, asegurando que las operaciones de lectura y escritura sobre las tablas **muestras**, **predicciones**, **análisis** y **calibraciones** mantienen la integridad referencial definida en el esquema.

Validación del modelo de IA y concurrencia

Se han llevado a cabo pruebas específicas sobre el flujo de inferencia. Por un lado, se ha validado que el modelo híbrido recibe los tensores con la dimensionalidad y normalización correctas, emitiendo predicciones consistentes con las métricas de precisión obtenidas durante su fase de entrenamiento. La selección de esta arquitectura final estuvo precedida por una fase de experimentación en la que se compararon múltiples algoritmos de aprendizaje automático (Random Forest mediante **scikit-learn** y XGBoost mediante **xgboost**, entre otros), evaluando su rendimiento sobre el conjunto de datos espectrales recogido con el prototipo físico. Esta fase empírica justificó la adopción del modelo definitivo, cuyos resultados quedan recogidos en el capítulo de aspectos relevantes de la memoria principal.

Por otro lado, se ha comprobado exhaustivamente el comportamiento de la interfaz gráfica durante el análisis concurrente, confirmando que la delegación de tareas pesadas a hilos **QThread** evita bloqueos en la GUI y permite una experiencia de usuario fluida y reactiva.

Análisis estático de calidad del código

Con el objetivo de evaluar de forma objetiva la calidad del código desarrollado, se ha integrado la herramienta **SonarQube**[23] en el repositorio del proyecto mediante el fichero de configuración `sonar_project.properties`.

La instancia utilizada es una instalación autoalojada en **Docker**, desplegada en un servidor local, lo que permite mantener el análisis dentro de la infraestructura local sin dependencias de servicios externos en la nube. El análisis se lanza de forma manual desde el equipo de desarrollo mediante **sonar-scanner**, autenticándose con el token de acceso personal generado en dicha instancia.

Esta herramienta realiza un análisis estático del código, detectando vulnerabilidades de seguridad, problemas de fiabilidad, deuda técnica de mantenibilidad y duplicaciones. El análisis inicial del código fuente superó el umbral de calidad (*Quality Gate: Passed*), con los siguientes resultados por categoría:

- **Seguridad:** 0 incidencias abiertas — calificación **A**.
- **Fiabilidad:** 0 incidencias abiertas — calificación **A**.
- **Mantenibilidad:** 49 incidencias abiertas — calificación **A**. Estas incidencias corresponden principalmente a sugerencias de refactorización menores (*code smells*) inherentes al carácter iterativo del desarrollo ágil, sin impacto funcional en el sistema.
- **Cobertura de pruebas:** 0,0 % sobre 8.800 líneas analizadas. SonarQube no recibía ningún informe de cobertura externo en esta configuración inicial, por lo que la métrica no refleja la batería de pruebas real del proyecto.
- **Duplicaciones:** 45,7 % sobre 8.800 líneas. Este índice elevado se explica por la coexistencia en el directorio **src/** de la codebase legada (**src/gui/**, **src/db/**) y el paquete activo refactorizado (**src/hives/**), que comparten estructuras equivalentes durante el periodo de transición arquitectónica.
- **Security Hotspots:** 0 — calificación **A**.

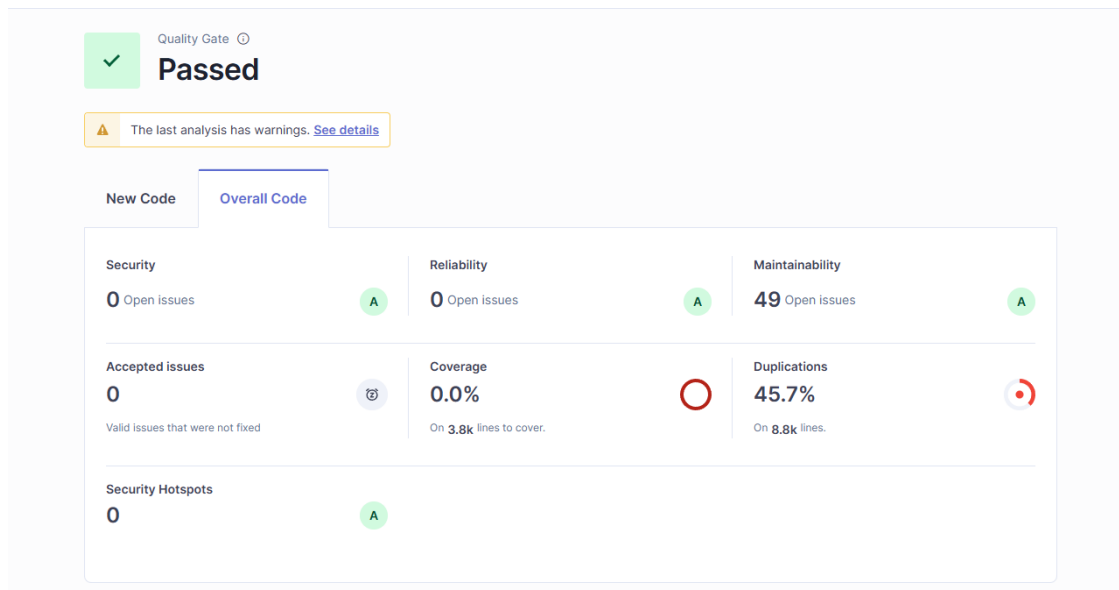


Figura D.1: Informe de calidad del código generado por SonarQube en la configuración inicial (`sonar.sources=src`). El proyecto supera el *Quality Gate* con calificación A en seguridad, fiabilidad y mantenibilidad; no obstante, la cobertura de pruebas registra 0,0 % por ausencia de informe de cobertura externo, y el índice de duplicaciones alcanza el 45,7 % debido a la presencia simultánea de código legado y código refactorizado en el repositorio. Fuente: Elaboración Propia.

Intentos de mejora de la integración con SonarQube

Tras analizar los resultados obtenidos, se identificaron dos limitaciones en la integración y se procedió a corregirlas experimentalmente.

Cobertura de pruebas. Se instrumentó la suite de pruebas con `pytest-cov` para generar un informe en formato Cobertura XML:

```
pytest --cov=src/hives --cov-report=xml:coverage.xml tests/
```

La ejecución confirmó una **cobertura real del 71,47 %** sobre 1.556 líneas del paquete activo (1.112 líneas cubiertas), con resultados destacados en los módulos críticos: `reports/pdf_report.py` (97,10 %), `core/database.py` (93,75 %) e `inference/model.py` (86,21 %). El fichero `coverage.xml` generado queda incluido en el repositorio como evidencia.

Sin embargo, a pesar de añadir la propiedad `sonar.python.coverage.reportPaths=coverage.xml` al fichero de configuración, la métrica de cobertura continuó mostrando 0,0 % en la interfaz de SonarQube. La causa más probable es una incompatibilidad entre la versión autoalojada de SonarQube y el formato del informe generado por la versión de `coverage.py` utilizada.

Duplicaciones y restricción del alcance del análisis. Para reducir el índice de duplicaciones, se modificó la propiedad `sonar.sources=src/hives` con el objetivo de restringir el análisis únicamente al paquete activo. No obstante, la ejecución produjo un resultado inesperado: el número de líneas analizadas se multiplicó desde las 8.800 iniciales hasta aproximadamente 212.000, y la calificación de **Fiabilidad descendió a C**, con más de 12.000 incidencias reportadas.

Este comportamiento indica que la ruta especificada incluyó inadvertidamente directorios de dependencias instaladas —presumiblemente el directorio `_internal` o el entorno virtual de Python presente en la misma jerarquía—, los cuales fueron indexados por SonarQube junto con el código propio. El elevado número de incidencias de fiabilidad es, por tanto, consecuencia del análisis de bibliotecas de terceros, y no refleja la calidad del código fuente desarrollado en el proyecto.

Ambas limitaciones quedan identificadas como líneas de mejora futura: la correcta configuración de `sonar.exclusions` para excluir dependencias y entornos virtuales, y la resolución de la incompatibilidad en el formato del informe de cobertura.

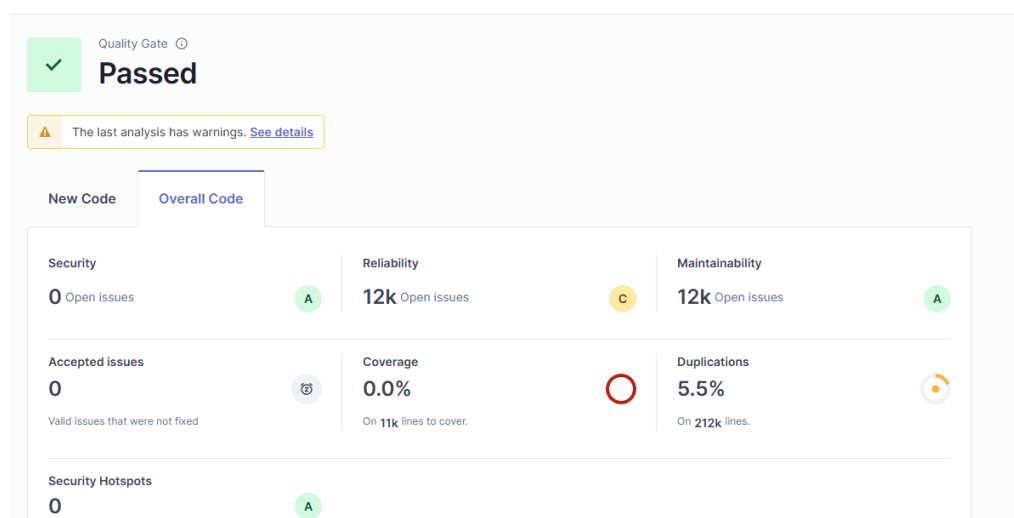


Figura D.2: Informe de SonarQube tras la modificación de `sonar.sources=src/hives`. El incremento no esperado de líneas analizadas (hasta 212.000) y el descenso de Fiabilidad a C evidencian que el analizador indexó dependencias externas incluidas en la ruta configurada, y no únicamente el código fuente del proyecto. Fuente: Elaboración Propia.

Compilación y empaquetado del ejecutable

La distribución de HIVES se realiza mediante **PyInstaller**[16], herramienta que genera un ejecutable autocontenido que encapsula el intérprete de Python, todas las dependencias y el modelo de Inteligencia Artificial, eximiendo al usuario final de cualquier instalación técnica previa. El proceso de empaquetado queda íntegramente definido en el fichero `HIVES.spec`, ubicado en la raíz del repositorio, que actúa como especificación reproducible del build. Para generar el ejecutable basta con ejecutar desde la raíz del repositorio:

```
pyinstaller HIVES.spec
```

El resultado se deposita en `dist/HIVES/`, siguiendo el formato *one-dir*: un ejecutable principal (`HIVES.exe` en Windows, `HIVES` en Linux/macOS) acompañado de un directorio `_internal/` que contiene los binarios, las DLLs y el modelo `mejor_modelo.keras`. Este formato fue seleccionado frente al *one-file* porque elimina la fase de auto-extracción al arranque, reduciendo el tiempo de inicio de 4–8 s a 2–4 s.

Optimizaciones aplicadas al spec. El fichero `HIVES.spec` incorpora un conjunto de optimizaciones progresivas orientadas a reducir el tiempo de build y el tiempo de arranque del ejecutable:

- **Exclusión de paquetes no utilizados.** Se declaran en el parámetro `excludes` los paquetes que PyInstaller rastrearía innecesariamente: `pandas`, `scipy`, `sklearn`, `cv2`, `setuptools`, `pdb`, `lib2to3`, `curses`, `idlelib` y `turtledemo`, entre otros. La aplicación únicamente requiere `PyQt6`, `TensorFlow`, `NumPy`, `matplotlib`, `FPDF2` y `PySerial`. Nótese que `PIL` y `unittest` no se excluyen porque `matplotlib 3.10+` los referencia internamente. Esta medida reduce el tiempo de la fase de análisis de PyInstaller en aproximadamente un 20–40 %.
- **Optimización de bytecode.** El parámetro `optimize=2` instruye a Python para que compile los módulos embebidos eliminando *docstrings* y aplicando optimizaciones de *peephole*. Esto reduce ligeramente el tamaño de los `.pyc` y acelera la carga de módulos en torno a un 5–10 %, sin ningún impacto en la funcionalidad.
- **Desactivación de UPX.** La compresión UPX (`upx=False`) ha sido deshabilitada en Windows y Linux. Si bien UPX reduce el tamaño del bundle, añade tiempo de compresión en el build y tiempo de descompresión en cada arranque, lo que penaliza la experiencia de uso en el contexto de este proyecto.
- **Runtime hook de TensorFlow.** Se ha creado el fichero de `src/runtime_hook.py`, referenciado en el parámetro `runtime_hooks` del spec, que se ejecuta antes de cualquier `import` de la aplicación. Establece tres variables de entorno críticas: `TF_CPP_MIN_LOG_LEVEL=3` suprime los mensajes de diagnóstico de TensorFlow en consola; `CUDA_VISIBLE_DEVICES=-1` impide la inicialización del subsistema CUDA/GPU, ahorrando entre 2 y 3 segundos de arranque en equipos sin tarjeta gráfica dedicada (perfil habitual del usuario objetivo) y evitando crashes por drivers ausentes; `OMP_NUM_THREADS=4` limita el paralelismo de OpenMP durante la carga de TensorFlow, evitando la proliferación de hilos competidores sobre la CPU.

Resolución del backend gráfico de matplotlib. Durante las pruebas del ejecutable empaquetado se detectó que el canvas espectral mostraba el mensaje «*matplotlib no disponible*». La causa raíz fue que PyInstaller no incluía el backend interactivo `QtAgg`, ya que su importación se produce

dentro de un bloque `try/except` que el analizador estático de PyInstaller no rastrea. La solución requirió tres intervenciones coordinadas:

1. Forzar la selección del backend antes de cualquier import, añadiendo `matplotlib.use('QtAgg')` al inicio del runtime hook.
2. Declarar explícitamente en `hiddenimports` los módulos `matplotlib.backends.backend_qtagg`, `backend_qt` y `backend_agg`, así como sus dependencias transitivas (`kiwisolver`, `pyparsing`, `contourpy`, `packaging`, `fonttools`, `dateutil`), completando el listado con `collect_submodules('matplotlib.backends')`.
3. Incluir los datos estáticos de matplotlib (fuentes y `matplotlibrc`) mediante `collect_data_files('matplotlib')` y un hook personalizado en `src/hooks/hook-matplotlib.py`.

El conjunto de optimizaciones aplicadas reduce el tiempo de build de los 2–5 minutos originales a aproximadamente 30–60 segundos, y el tiempo de arranque de 4–8 segundos a 2–4 segundos, manteniendo el 100 % de la funcionalidad del sistema.

Apéndice *E*

Documentación de usuario

E.1. Introducción

El presente apéndice constituye la documentación orientada al usuario final del sistema HIVES (*Honey Identification and Verification by Spectroscopy*). Su objetivo es proporcionar las instrucciones necesarias para la correcta instalación, configuración y operación de la aplicación, sin requerir conocimientos de programación por parte del usuario.

El sistema ha sido diseñado para ser operado por dos perfiles de usuario diferenciados:

- **Usuario técnico o semitécnico:** personal de laboratorio, técnicos de calidad apícola o investigadores con conocimientos básicos en el uso de un ordenador. Este perfil es capaz de gestionar la configuración del puerto serie y la calibración del sensor de forma autónoma.
- **Usuario no técnico:** apicultores u operarios sin conocimientos informáticos avanzados, cuya interacción con el sistema se limita al flujo principal de análisis. Para este perfil se recomienda que la configuración inicial sea realizada por personal técnico.

La interfaz gráfica sigue principios de diseño minimalista, minimizando la curva de aprendizaje y permitiendo obtener resultados de clasificación de forma intuitiva y eficiente.

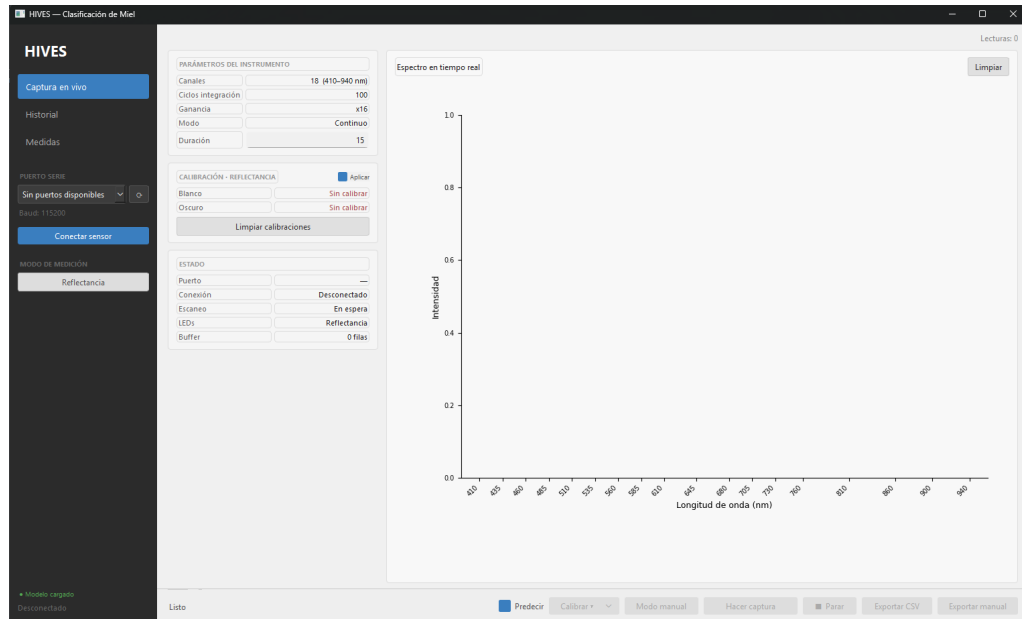


Figura E.1: Vista general de la ventana principal de la aplicación HIVES.
Fuente: Elaboración Propia.

E.2. Requisitos de usuario

Para garantizar el correcto funcionamiento del sistema HIVES, el equipo sobre el que se ejecute la aplicación deberá cumplir los siguientes requisitos mínimos de hardware y software según el sistema operativo empleado:

Componente	Especificación mínima
Sistema operativo	Windows 10 (64 bits) o superior; Ubuntu 20.04 LTS o superior; macOS 11 (<i>Big Sur</i>) o superior
Procesador	Intel Core i3 (o equivalente AMD), 2,0 GHz, o ARM (M1, M2,...) en el caso de macOS
Memoria RAM	4 GB
Almacenamiento	2 GB de espacio libre en disco
Puerto de conexión	Puerto USB libre (para adaptador USB–Serie)
Conectividad	No requerida (operación completamente local)

Tabla E.1: Requisitos mínimos del sistema para la ejecución de HIVES.

Adicionalmente, es necesario que el espectrómetro se encuentre conectado al equipo mediante el adaptador USB–Serie correspondiente y que el sistema operativo haya reconocido correctamente el dispositivo antes de iniciar la aplicación.

E.3. Instalación

La distribución de HIVES se realiza en forma de paquete autocontenido generado mediante la herramienta PyInstaller en formato *one-dir*. Dicho formato encapsula el intérprete de Python, todas las dependencias y el modelo de Inteligencia Artificial, eximiendo al usuario de cualquier instalación técnica previa. El resultado es una carpeta de aplicación (HIVES/) que contiene el ejecutable principal junto a un subdirectorio `_internal/` con las bibliotecas, los binarios de soporte y el modelo entrenado. Se proporciona un paquete independiente para cada sistema operativo compatible.

Importante: la carpeta de aplicación debe copiarse y conservarse *íntegra*. El ejecutable principal depende del subdirectorio `_internal/` que lo acompaña; si se extrae el ejecutable de forma aislada, la aplicación no podrá iniciarse. En macOS esta restricción no aplica, ya que la distribución adopta la forma de un *bundle .app* autocontenido.

Windows

1. Copiar la carpeta completa `HIVES\` en el directorio de trabajo deseado (por ejemplo, `C:\Usuarios\<usuario>\HIVES\`), conservando en su interior el ejecutable `HIVES.exe` y el subdirectorio `_internal\`.
2. Conectar el espectrómetro al ordenador mediante el cable USB antes de iniciar la aplicación. El sensor se comunica con el equipo a través de la placa Arduino Uno que lo gobierna. Verificar en el *Administrador de dispositivos* que el dispositivo aparece reconocido y que se le ha asignado un puerto COM activo (por ejemplo, `COM3`).
3. Hacer doble clic sobre `HIVES.exe` para iniciar la aplicación. No se requiere instalación adicional ni permisos de administrador.
4. Asegurarse de que la carpeta donde reside la aplicación cuenta con permisos de escritura, ya que la base de datos local y los reportes exportados se almacenan en esa misma ubicación.

Linux

1. Copiar la carpeta completa `HIVES/` en el directorio de trabajo deseado, conservando en su interior el binario `HIVES` y el subdirectorio `_internal/`.
2. Conectar el espectrómetro al ordenador mediante el cable USB. La placa Arduino que gobierna el sensor se enumera en Linux con un nombre de dispositivo que depende de su circuito de comunicación: las placas con conversor USB-serie (CH340, FTDI o CP210x) aparecen como `/dev/ttyUSB0`, mientras que las placas con USB nativo lo hacen como `/dev/ttyACM0`. Para conocer el puerto asignado, listar ambos conjuntos de dispositivos:

```
ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
```

Si hubiera varios dispositivos conectados y existiera duda sobre cuál corresponde al espectrómetro, puede desconectarse y volverse a conectar la placa y consultar los últimos mensajes del núcleo, que indican el nombre exacto recién asignado:

```
dmesg | grep -i tty
```

3. Conceder al usuario permiso de acceso al puerto serie. Los dispositivos `ttyUSB` y `ttyACM` pertenecen al grupo `dialout`, por lo que basta con añadir el usuario a dicho grupo (solución persistente y recomendada) y conceder permiso de ejecución al binario:

```
chmod +x HIVES/HIVES
sudo usermod -aG dialout $USER
```

Es necesario cerrar la sesión y volver a iniciarla para que el cambio de grupo surta efecto. Como alternativa puntual para una única sesión, puede otorgarse acceso directo al dispositivo concreto (sustituyendo el nombre por el identificado en el paso anterior):

```
sudo chmod a+rw /dev/ttyUSB0
```

4. Ejecutar la aplicación desde el terminal, situándose en la carpeta de la aplicación, o haciendo doble clic sobre el binario si el entorno de escritorio lo permite:

```
cd HIVES && ./HIVES
```

macOS (Apple Silicon, M1 o superior)

1. Copiar el paquete `HIVES.app` en la carpeta *Aplicaciones* o en el directorio de trabajo deseado.

Nota: el ejecutable distribuido ha sido compilado para la arquitectura ARM64 (Apple Silicon). No ha sido probado en otras arquitecturas. Para equipos Mac con procesador Intel, se requerirá el uso de Rosetta 2.

2. Conectar el espectrómetro al ordenador mediante el cable USB. El sistema operativo reconocerá automáticamente la placa Arduino Uno y le asignará un puerto con un nombre similar a `/dev/cu.usbmodem1401`, lo que puede verificarse desde la terminal con:

```
ls /dev/cu.*
```

3. En caso de que macOS bloquee la ejecución por provenir de un desarrollador no identificado, acceder a *Ajustes del Sistema* → *Privacidad y seguridad* y autorizar manualmente la ejecución de la aplicación.
4. Abrir la aplicación haciendo doble clic sobre `HIVES.app`.

E.4. Manual del usuario

Esta sección describe el flujo de trabajo completo de la aplicación HIVES, detallando las pantallas disponibles y las acciones que el usuario puede realizar en cada una de ellas.

Estructura de la interfaz

La aplicación está organizada en tres elementos principales:

- **Menú lateral izquierdo:** panel de control desde el que se gestiona la conexión con el Arduino Uno, se selecciona el puerto serie activo y se accede a los parámetros de configuración del análisis.
- **Ventana principal:** área central de operación donde se inicia el análisis, se visualizan los resultados y se exportan los reportes.
- **Ventana de históricos:** repositorio de consulta de todos los análisis previos registrados, con capacidad de filtrado por fecha, categoría y nivel de confianza. Además se pueden extraer los datos a un fichero CSV.
- **Ventana de muestras:** repositorio de consulta de todas las muestras tomadas hasta la fecha, con capacidad de filtrado por fecha, y de extracción de las mismas en CSV.

Paso 1 — Configuración inicial (perfil técnico)

Este paso está destinado al usuario técnico. El usuario no técnico puede omitirlo si la configuración ya ha sido realizada previamente.

1. En el **menú lateral izquierdo** de la ventana principal, localizar el selector de puerto serie.
2. Seleccionar el puerto al que está conectado el Arduino Uno. La lista se actualiza automáticamente con los puertos disponibles detectados por el sistema operativo.
3. Ajustar los parámetros de análisis adicionales si fuera necesario. La aplicación conservará esta configuración en sesiones sucesivas.

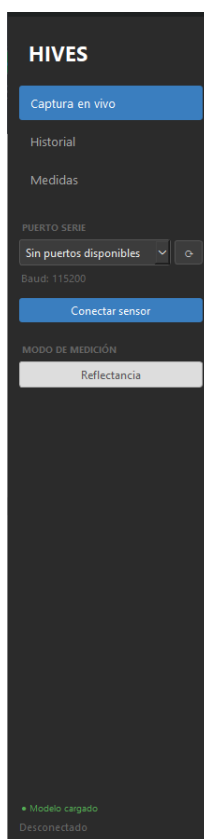


Figura E.2: Menú lateral izquierdo de la ventana principal: selector de puerto serie y parámetros de análisis. Fuente: Elaboración Propia.

Paso 2 — Calibración del sensor (perfil técnico)

Se recomienda realizar una calibración del sensor al inicio de cada sesión de trabajo, especialmente si el dispositivo ha sido desconectado o si las condiciones ambientales han variado respecto a la sesión anterior.

1. Desde la ventana principal, accionar el control de **Calibración**.
2. Seguir las indicaciones que aparezcan en pantalla durante el proceso. Habrá que hacer 2 calibraciones: una de blancos y otra de negros. Para ello se usará un papel de reflectancia en el primer caso, y una cubeta negra en el segundo caso.
3. Una vez completada la calibración, el sistema estará listo para realizar análisis con garantías de precisión.

Paso 3 — Análisis de una muestra

Este es el flujo principal de uso de la aplicación, accesible para cualquier perfil de usuario.

1. Colocar la muestra de miel en el soporte del espectrómetro conforme al procedimiento físico establecido.
2. En la **ventana principal**, accionar el control «**Iniciar Análisis**».
3. El sistema establecerá la conexión con el espectrómetro, capturará los datos espectrales y ejecutará el modelo de clasificación de forma automática. Durante este proceso se mostrará un indicador de progreso en pantalla.
4. Transcurrido el tiempo de análisis (máximo 2 minutos), los resultados se visualizarán en la interfaz: **categoría de miel clasificada**, **índice de confianza del modelo** y **métricas espectrales** de la muestra.
5. Los datos quedan registrados automáticamente en la base de datos local sin intervención del usuario.

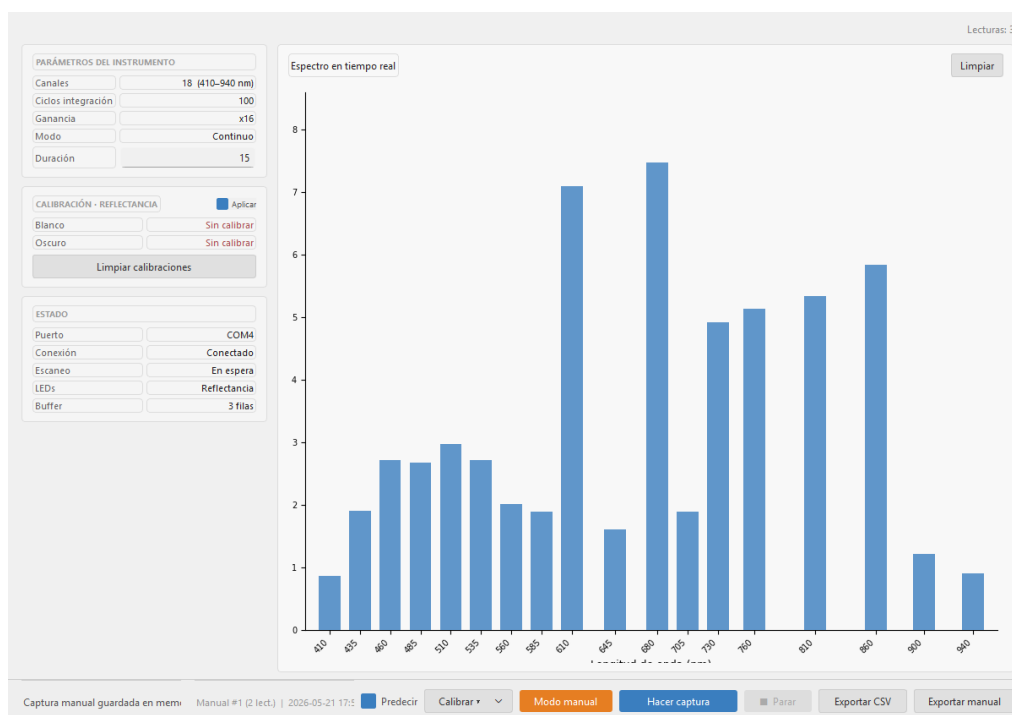


Figura E.3: Ventana principal mostrando los resultados tras completar un análisis de muestra. Fuente: Elaboración Propia.

Paso 4 — Exportación del reporte PDF

1. Accionar el control «**Generar Reporte PDF**» en la ventana principal.
2. El sistema abrirá un cuadro de diálogo del sistema operativo para seleccionar la ruta y el nombre del archivo de destino.
3. Confirmar la ruta. El documento PDF se generará y guardará en el directorio indicado, quedando disponible para su consulta, impresión o distribución externa.

Paso 5 — Consulta del histórico (perfil técnico)

1. Accionar el control «**Ver Histórico**» en la ventana principal.
2. La **ventana de históricos** mostrará una tabla con todos los análisis registrados: fecha, tipo de miel clasificado y nivel de confianza.

- 3. Utilizar los controles de filtrado disponibles (rango de fechas, categoría de miel, umbral de confianza) para acotar los resultados.
- 4. Seleccionar un registro de la tabla para visualizar su detalle completo o iniciar la exportación PDF descrita en el paso anterior.

Historial de datos

Análisis guardados en la base de datos.

	ID	Nombre	Timestamp	Modo	Calibrado	Clase	
1	31	1	2026-05-13 14:12:57	transmitancia	Sí	Clase_3	
2	30	1	2026-05-13 14:12:26	transmitancia	Sí	Clase_3	
3	29	1	2026-05-13 14:11:43	reflectancia	Sí	Clase_3	
4	28	1	2026-05-13 14:11:09	reflectancia	Sí	Clase_3	
5	27	1	2026-05-13 14:10:48	reflectancia	Sí	Clase_3	
6	26	1	2026-05-13 14:08:49	reflectancia	Sí	Clase_3	

Figura E.4: Ventana de históricos con los filtros de búsqueda disponibles.
Fuente: Elaboración Propia.

Mensajes de error frecuentes

Mensaje	Acción recomendada
«No se detecta el espectrómetro»	Verificar que el cable USB del Arduino Uno está correctamente conectado. Comprobar que el puerto serie seleccionado en la ventana de configuración coincide con el asignado por el sistema operativo (COMx en Windows, /dev/ttyACMO en Linux, /dev/cu.usbmodem* en macOS). En Linux, verificar además que el usuario pertenece al grupo dialout.
«Error al cargar el modelo de IA»	Comprobar que el archivo del modelo (mejor_modelo.keras) y el fichero de clases (clases.json) se encuentran en la carpeta models/ junto al ejecutable.
«Sin permisos de escritura»	Mover el ejecutable a un directorio con permisos de escritura para el usuario actual. En macOS y Linux, verificar los permisos del directorio con <code>ls -la</code> .
«No se han encontrado coincidencias»	Ampliar los criterios de filtrado en la ventana de históricos.

Tabla E.2: Mensajes de error de la aplicación y acciones correctoras.

Apéndice F

Sostenibilización curricular

F.1. Introducción

El presente apéndice analiza el Trabajo de Fin de Grado desde la perspectiva de la sostenibilidad, tomando como referencia los **Objetivos de Desarrollo Sostenible (ODS)** de la Agenda 2030 aprobada por la Organización de las Naciones Unidas [15], en línea con las directrices de sostenibilización curricular promovidas por CRUE Universidades Españolas [5]. Dado que el sistema HIVES se inscribe en el ámbito agroalimentario y se vincula directamente con el sector apícola, el análisis de su contribución a los objetivos resulta especialmente natural y verificable, lo que permite una valoración fundamentada y evitando declaraciones genéricas. En las secciones siguientes se identifican los objetivos sobre los que el proyecto ejerce una contribución directa, se detallan aquellos en los que influye y se examina, sin omitir sus límites, el impacto del propio proceso de desarrollo.

F.2. Contribución a los Objetivos de Desarrollo Sostenible

La Agenda 2030 articula su llamamiento a la acción en torno a 17 Objetivos de Desarrollo Sostenible interrelacionados, que integran de manera equilibrada las dimensiones social, económica y ambiental del desarrollo [15]. La Figura F.1 recoge el conjunto completo de objetivos que conforman dicho marco de referencia.



Figura F.1: Los 17 Objetivos de Desarrollo Sostenible de la Agenda 2030 de Naciones Unidas. Fuente: [2]

El sistema HIVES no tiene como objetivo el influir sobre todos los objetivos, planteamiento que transicionaría el análisis hacia irrelevante. Por el contrario, se identifican **cuatro objetivos** sobre los que el proyecto ejerce una contribución directa y demostrable, así como un conjunto adicional de objetivos sobre los que incide de manera transversal.

Objetivos abordados de forma directa

La aportación del proyecto se concentra en cuatro objetivos estrechamente ligados a la naturaleza agroalimentaria y tecnológica de la solución, recogidos en la Figura F.2 y desarrollados a continuación en relación con sus metas específicas.

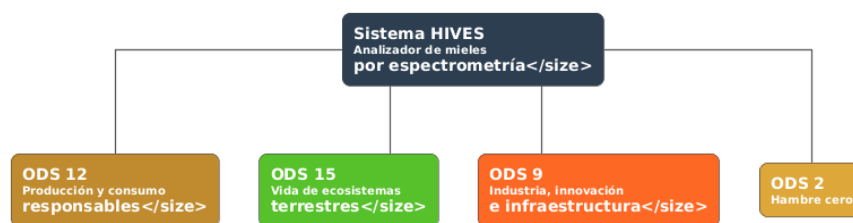


Figura F.2: Objetivos de Desarrollo Sostenible sobre los que el sistema HIVES ejerce una contribución directa. Fuente: Elaboración Propia.

ODS 12 — Producción y consumo responsables. Constituye el objetivo más directamente vinculado al proyecto. Su **meta 12.8** persigue garantizar que las personas dispongan de la información pertinente para adoptar decisiones de consumo responsables. El fraude alimentario y el etiquetado engañoso de la miel rompen precisamente esa cadena de información, al desligar el precio y la etiqueta del origen botánico real del producto. Al automatizar la verificación de dicho origen, HIVES refuerza la **trazabilidad y la autenticidad** del producto, restituyendo al consumidor una información veraz y reforzando la confianza en el mercado. La solución incide igualmente en la meta 12.6, en tanto que dota a los agentes productores de una herramienta para acreditar la calidad y la integridad de sus productos.

ODS 15 — Vida de ecosistemas terrestres. Su **meta 15.5** aborda la adopción de medidas para reducir la degradación de los hábitats naturales y frenar la pérdida de biodiversidad. La conexión del proyecto con este objetivo es de naturaleza económica e indirecta, pero sólida: la viabilidad comercial de la apicultura constituye un incentivo decisivo para el mantenimiento de las colmenas y, con ellas, de las poblaciones de abejas, agentes polinizadores esenciales para la reproducción de numerosas especies vegetales y para el sostenimiento de los ecosistemas. Al proteger el valor diferencial de las mieles monoflorales frente a la adulteración, HIVES contribuye a preservar la rentabilidad de la actividad de los apicultores.

ODS 9 — Industria, innovación e infraestructura. Su **meta 9.5** promueve el aumento de la investigación científica y la mejora de la capacidad tecnológica. El proyecto materializa una innovación aplicada que **democratiza** el acceso a una técnica de análisis avanzada: mediante la fusión de espectrometría óptica de bajo coste e inteligencia artificial, traslada una capacidad analítica antes reservada a laboratorios especializados a una solución autónoma de escritorio. Esta reducción de la barrera tecnológica es,

en sí misma, una contribución al desarrollo de infraestructuras de calidad, fiables y asequibles.

ODS 2 — Hambre cero. Más allá de la mera disponibilidad de alimentos, su **meta 2.4** incide en la sostenibilidad y la fiabilidad de los sistemas de producción agroalimentaria. La garantía de autenticidad y calidad de un producto de alto valor nutricional como la miel se alinea con la promoción de cadenas alimentarias transparentes y sostenibles, en las que la información del producto se corresponde con sus características reales.

Objetivos abordados de forma transversal

Junto a los objetivos anteriores, el proyecto presenta vínculos secundarios con otros ODS que, sin constituir su eje central, se ven favorecidos por su desarrollo. Así, la defensa de la competencia leal y de los medios de vida de los productores honestos frente a las prácticas adulteradoras conecta con el **ODS 8** (trabajo decente y crecimiento económico); la detección de adulteraciones que pudieran comprometer las propiedades del producto destinado al consumo enlaza con el **ODS 3** (salud y bienestar); y el propio Trabajo de Fin de Grado, en tanto que proceso formativo, ejercita competencias transversales asociadas al **ODS 4** (educación de calidad).

La Tabla F.1 consolida la totalidad de las contribuciones, asociando cada objetivo con la evidencia concreta que la sustenta dentro del proyecto.

F.3. Análisis del impacto del proyecto por dimensiones

Con el fin de complementar el análisis anterior y evitar una valoración exclusivamente propositiva, se examina el proyecto conforme a las tres dimensiones clásicas de la sostenibilidad, contemplando tanto sus aportaciones como sus costes reales. La Figura F.3 ilustra dicha intersección.

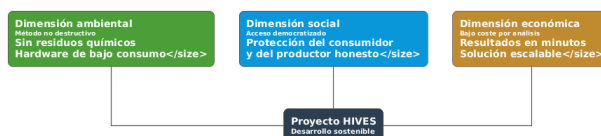


Figura F.3: Las tres dimensiones de la sostenibilidad y su intersección con el proyecto HIVES. Fuente: Elaboración Propia.

ODS	Objetivo	Contribución del proyecto HIVES
12	Producción y consumo responsables	Verificación automática de la autenticidad y el origen botánico; refuerzo de la trazabilidad y lucha contra el fraude alimentario (metas 12.8 y 12.6).
15	Vida de ecosistemas terrestres	Protección del valor económico de la apicultura como incentivo para la conservación de las abejas polinizadoras y la biodiversidad (meta 15.5).
9	Industria, innovación e infraestructura	Innovación que democratiza una técnica analítica avanzada mediante hardware de bajo coste e inteligencia artificial (meta 9.5).
2	Hambre cero	Garantía de calidad y autenticidad en un producto agroalimentario de alto valor, en favor de sistemas de producción fiables (meta 2.4).
8	Trabajo decente y crecimiento económico	Defensa de la competencia leal y de los medios de vida de los productores honestos frente a las prácticas adulteradoras.
3	Salud y bienestar	Detección de adulteraciones que podrían comprometer las propiedades del producto destinado al consumo.
4	Educación de calidad	El propio TFG, en tanto que proceso formativo, ejercita competencias transversales en sostenibilidad.

Tabla F.1: Correspondencia entre los Objetivos de Desarrollo Sostenible y las aportaciones del proyecto HIVES.

Dimensión ambiental. El balance ambiental es marcadamente favorable. El método de análisis es **no destructivo** y **libre de reactivos**, lo que elimina el consumo de muestras y la generación de residuos químicos propios del procedimiento melisopalinológico tradicional. En aras del rigor, debe reconocerse que la fase de entrenamiento de los modelos conllevó un coste computacional no despreciable derivado del uso de Google Colab Pro; dicho impacto queda acotado por tratarse de una tarea puntual y no recurrente, y por desplegarse en producción un modelo ligero que infiere localmente sobre CPU sin dependencia de la nube.

Dimensión social. La principal aportación es la **democratización** del acceso a una capacidad analítica antes ligada a instrumental avanzado y a personal especializado en botánica. Al reducir la barrera técnica y económica, el sistema habilita a pequeños productores y cooperativas para verificar la autenticidad de su producción, contribuyendo a un mercado más transparente y equitativo, a la vez que protege al consumidor frente al fraude.

Dimensión económica. La viabilidad económica, detallada en el Apéndice A, respalda la sostenibilidad a largo plazo de la propuesta. Frente al elevado coste por muestra y a los tiempos de días o semanas del análisis de referencia, el sistema ofrece un cribado de coste marginal muy reducido y resultados en cuestión de minutos, con un coste total de desarrollo estimado en 3.247,68 € y una arquitectura plenamente **escalable**.

F.4. Reflexión ética y profesional

El desarrollo de una herramienta de verificación de la autenticidad de un producto alimentario conlleva una responsabilidad deontológica que el proyecto asume desde tres principios. El primero es la **honestidad metodológica**: la documentación expone con transparencia las limitaciones del conjunto de datos, el recurso al aumento sintético de muestras y las métricas reales alcanzadas, sin presentar el prototipo como un sustituto con validez legal del método normalizado de referencia. El segundo es la **minimización y custodia de datos**: la información se persiste de forma local, sin transferencias a terceros, en coherencia con la viabilidad legal analizada en el Apéndice A. El tercero es la **orientación al bien común**: la finalidad última de la herramienta es proteger tanto al consumidor como al productor honesto, situando la tecnología al servicio de la confianza en la cadena agroalimentaria.

F.5. Conclusión

El análisis evidencia que HIVES no constituye un desarrollo técnicamente neutro, sino una solución con un impacto positivo articulado y verificable sobre la sostenibilidad. Su contribución directa a los Objetivos de Desarrollo Sostenible 2, 9, 12 y 15, junto con su incidencia transversal sobre otros objetivos de la Agenda 2030, inscribe el proyecto de pleno en el marco de la sostenibilización curricular, integrando la responsabilidad ética y la conciencia de la huella propia como componentes inseparables de la práctica profesional del ingeniero.

Bibliografía

- [1] ams OSRAM AG. AS7265x spectral triad sensor datasheet. Technical report, ams AG, 2018. <https://ams.com/as7265x>. Consultado: junio 2026.
- [2] AVV Barrio Jesús. Ods (objetivos de desarrollo sostenible), 2026. [Accedido: 3-junio-2026].
- [3] Yoshua Bengio, Nicolas Boulanger-Lewandowski, and Razvan Pascanu. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013. <https://arxiv.org/abs/1211.5063>.
- [4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. <https://arxiv.org/abs/1603.02754>.
- [5] CRUE Universidades Españolas. Directrices para la introducción de la sostenibilidad en el currículum. <https://www.crue.org>, 2012. Documento del Grupo de Trabajo de Sostenibilización Curricular. Consultado: junio 2026.
- [6] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, MA, 2002. Capítulo 12: Serialized LOB pattern.
- [7] Martin Fowler. Mocks aren't stubs. <https://martinfowler.com/articles/mocksArentStubs.html>, 2007. martinfowler.com. Consultado: junio 2026.

- [8] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Boston, MA, 1994.
- [9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. <https://www.deeplearningbook.org>.
- [10] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, NY, 2 edition, 2009.
- [11] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. <https://arxiv.org/abs/1503.02531>.
- [12] Instituto Nacional de Estadística. Encuesta anual de estructura salarial (eas). año 2024. datos definitivos, 2026. Instituto Nacional de Estadística (INE), España.
- [13] Chris Liechti. pySerial documentation. <https://pyserial.readthedocs.io/en/latest/>, 2023. Consultado: junio 2026.
- [14] Ministerio de Inclusión, Seguridad Social y Migraciones. Tipos de cotización a la Seguridad Social vigentes. <https://www.seg-social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/10721>, 2026. Gobierno de España. Consultado: junio 2026.
- [15] Organización de las Naciones Unidas. Transformar nuestro mundo: la agenda 2030 para el desarrollo sostenible. <https://www.un.org/sustainabledevelopment/es/>, 2015. Resolución A/RES/70/1. Consultado: junio 2026.
- [16] PyInstaller Development Team. PyInstaller manual. <https://pyinstaller.org/en/stable/>, 2024. Consultado: junio 2026.
- [17] pytest Development Team. pytest documentation. <https://docs.pytest.org/en/stable/>, 2024. Consultado: junio 2026.
- [18] Qt Group. QThread class — Qt 6 documentation. <https://doc.qt.io/qt-6/qthread.html>, 2024. Consultado: junio 2026.
- [19] RAPIDS Development Team. cuML: GPU-accelerated machine learning library. <https://github.com/rapidsai/cuml>, 2023.

- [20] Riverbank Computing Limited. PyQt6 reference guide: Licence. <https://www.riverbankcomputing.com/commercial/pyqt>, 2024. Consultado: mayo de 2026.
- [21] Riverbank Computing Limited. PyQt6 reference guide: Signals and slots. https://www.riverbankcomputing.com/static/Docs/PyQt6/signals_slots.html, 2024. Consultado: junio 2026.
- [22] Ken Schwaber and Jeff Sutherland. The scrum guide. the definitive guide to scrum: The rules of the game. <https://scrumguides.org/scrum-guide.html>, 2020. Scrum.org. Consultado: junio 2026.
- [23] SonarSource. SonarQube documentation. <https://docs.sonarsource.com/sonarqube/>, 2024. Consultado: junio 2026.
- [24] SparkFun Electronics. SparkFun triad spectroscopy sensor – AS7265x (Qwiic). <https://www.sparkfun.com/products/15050>, 2023. Consultado: junio 2026.
- [25] SQLite Consortium. Atomic commit in SQLite (propiedades ACID). <https://www.sqlite.org/atomiccommit.html>, 2024. Consultado: junio 2026.
- [26] SQLite Consortium. SQLite: When to use SQLite. <https://www.sqlite.org/whentouse.html>, 2024. Consultado: junio 2026.